Real-Time Project

Building an End-to-End Healthcare AI Assistant with Microsoft Foundry

Test this real-time project to learn how to build an enterprise chatbot using Microsoft Foundry and Azure Portal with your own data. Ideal for AI Engineers, Data Engineers, and Cloud Professionals who want hands-on experience implementing Retrieval-Augmented Generation (RAG), configuring search indexing, deploying GPT models, and integrating custom datasets for contextual, citation-backed responses in Azure.

Scenario:

Epam aims to improve customer support with an AI-driven chatbot that delivers relevant information quickly and efficiently. By integrating Azure OpenAI with Azure Cognitive Search, the chatbot can access and process the company's proprietary data, allowing for precise responses to customer queries.

Description:

This chatbot utilizes Azure OpenAI to understand complex queries and Azure Cognitive Search to retrieve data from internal knowledge bases. Such a setup ensures users receive accurate, context-rich answers in real-time, improving customer satisfaction and operational efficiency at an enterprise scale.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

Contents

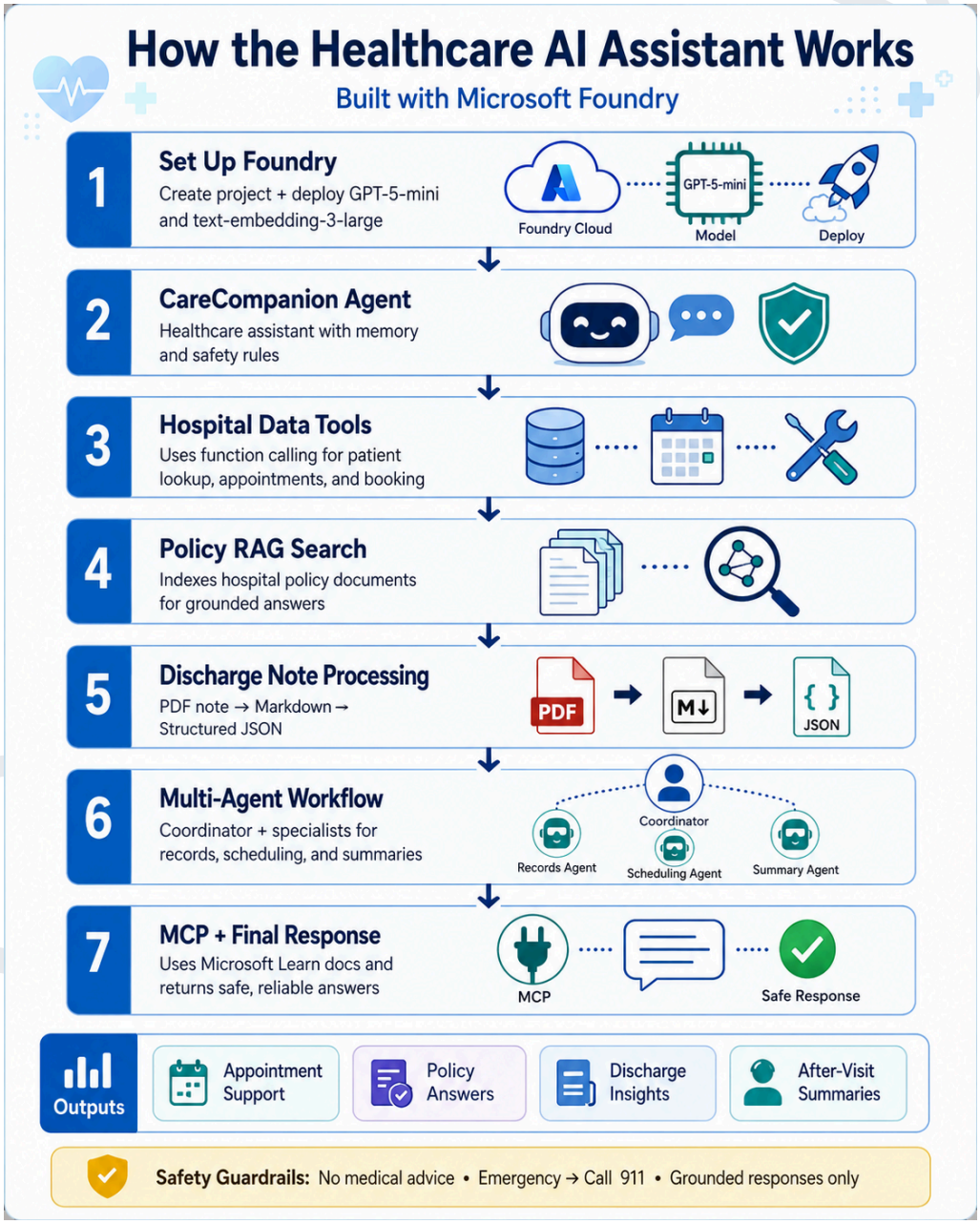
1. Introduction **3**
2. Overview and Key Concepts **5**
2.1 Microsoft AI Foundry 5
2.2 GPT-5-mini model 5
2.3 Text Enbedding 3 large model 5
2.4 Retrieval-Augmented Generation (RAG) 6
2.5 Multi-agent Orchesteration 6
3. Documentation Links **7**
4. Pre-requisite **8**
5. Cost to Perform the lab **11**
6. Setup Microsoft Foundry and deploy the AI model **12**
7. Setup the data in VS Code **17**
8. Build the Healthcare AI Assistant **23**
9. Integrate Hospital data using function calling **28**
10. Build the Multi-agent Healthcare Workflow **32**
11. Configure MCP **39**
12. Share Your Learning on LinkedIn & Community **42**
12.1 Share wins on LinkedIn 43
12.2 Share the win on the Community via Progress Diary 46
13. Delete Azure resources **47**
14. Troubleshooting Section **50**
15. Summary **51**

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

1. Introduction

In this Microsoft Foundry project, we build an end-to-end healthcare AI assistant that helps a fictional hospital manage routine patient-service requests, appointment information, hospital policies, and discharge documents.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

The solution includes several key stages:

- **Foundry Setup:** We create a Microsoft Foundry project, deploy a generative AI model, configure the Python environment, and test the first model call.
- **Healthcare AI Agent:** We build an AI assistant with conversation memory, hospital-specific instructions, emergency escalation rules, and restrictions against providing medical advice.
- **Function Calling:** The assistant is connected to synthetic patient and appointment data through Python tools, allowing it to look up records and manage follow-up appointments.
- **Retrieval-Augmented Generation:** Hospital policy documents are added to a vector store so the assistant can answer questions using verified information and provide relevant citations.
- **Document Understanding:** Azure AI Content Understanding is used to process discharge notes and extract important information into structured formats.
- **Multi-Agent Orchestration:** Multiple specialist agents are coordinated to handle patient records, hospital policies, discharge information, and safety-related requests.
- **Safety and Monitoring:** MCP tools, healthcare guardrails, tracing, monitoring, and evaluation are added to improve the reliability and safety of the complete solution.

The result is an intelligent healthcare assistant that can answer hospital-related questions, retrieve appointment details, process discharge documents, and coordinate different AI agents while following strict safety rules.

This project demonstrates how Microsoft Foundry, the Azure AI Projects SDK, Microsoft Agent Framework, RAG, Content Understanding, and MCP can be combined to build a practical healthcare-focused AI solution.

It takes approximately **3 to 4 hours** to complete all the sections and provides hands-on experience in building, testing, securing, and monitoring an end-to-end agentic AI application.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

2. Overview and Key Concepts

In this section, we'll cover key concepts used in this guide, including an overview of what's covered in this guide.

2.1 Microsoft AI Foundry

Microsoft AI Foundry is a cloud-based platform on Azure for building, testing, deploying, and managing generative AI applications and AI agents.

It provides access to foundation models, agent services, prompt management, evaluation, monitoring, safety controls, and project resources within a centralized development environment.

In this project, Microsoft AI Foundry is used to deploy models, create the healthcare assistant, manage conversations, integrate tools, and monitor the complete AI solution.

2.2 GPT-5-mini model

GPT-5-mini is a lightweight generative AI model designed to provide fast and cost-efficient responses for conversational and task-based applications.

It can understand user queries, follow instructions, generate natural-language responses, and support tool calling and agent-based workflows.

In this project, the model acts as the reasoning engine behind the healthcare assistant and specialist agents.

2.3 Text Enbedding 3 large model

The text-embedding-3-large model converts text into numerical vectors called embeddings, which represent the meaning and relationships within the content.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

These embeddings allow the system to compare user questions with stored documents and identify the most relevant information based on semantic similarity rather than exact keyword matches.

In this project, the embedding model supports document indexing and retrieval for hospital policy documents.

2.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is an AI approach that retrieves relevant information from external documents before generating a response.

It helps reduce unsupported answers by grounding the model's response in trusted organizational data, such as policies, manuals, and internal knowledge bases.

In this project, RAG enables the healthcare assistant to answer questions about visiting hours, billing, and discharge procedures using approved hospital documents with citations.

2.5 Multi-agent Orchesteration

Multi-agent orchestration is the process of coordinating multiple specialized AI agents to complete a task together.

Each agent can be assigned a specific responsibility, such as retrieving patient records, searching hospital policies, processing discharge documents, or applying safety rules.

In this project, a coordinator agent routes requests to the appropriate specialist agents and combines their outputs into a single, complete response.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

3. Documentation Links

1. What is Microsoft Foundry?
   https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry?tabs=python

2. What is Foundry Local?
   https://learn.microsoft.com/en-us/azure/foundry-local/what-is-foundry-local

3. Azure OpenAI reasoning models
   https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/reasoning?tabs=csharp%2Cgpt-5

4. Retrieval-augmented generation (RAG) in Azure AI Search
   https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview?tabs=videos

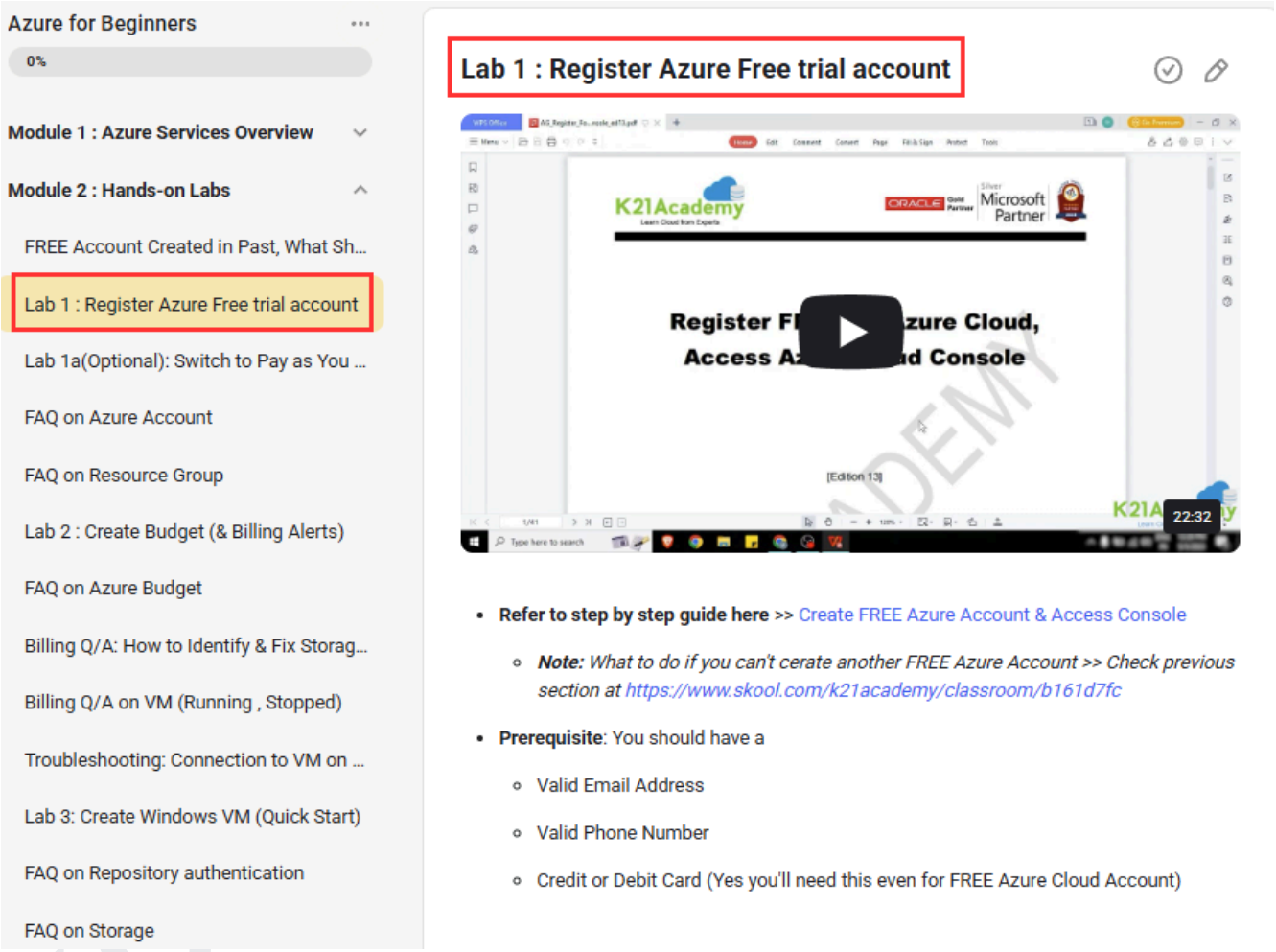
5. Design and develop a RAG solution
   https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide

6. Multi-Agent Orchestration with Azure AI Foundry
   https://techcommunity.microsoft.com/blog/azureinfrastructureblog/multi%E2%80%91agent-orchestration-with-azure-ai-foundry-from-idea-to-production/4449925

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

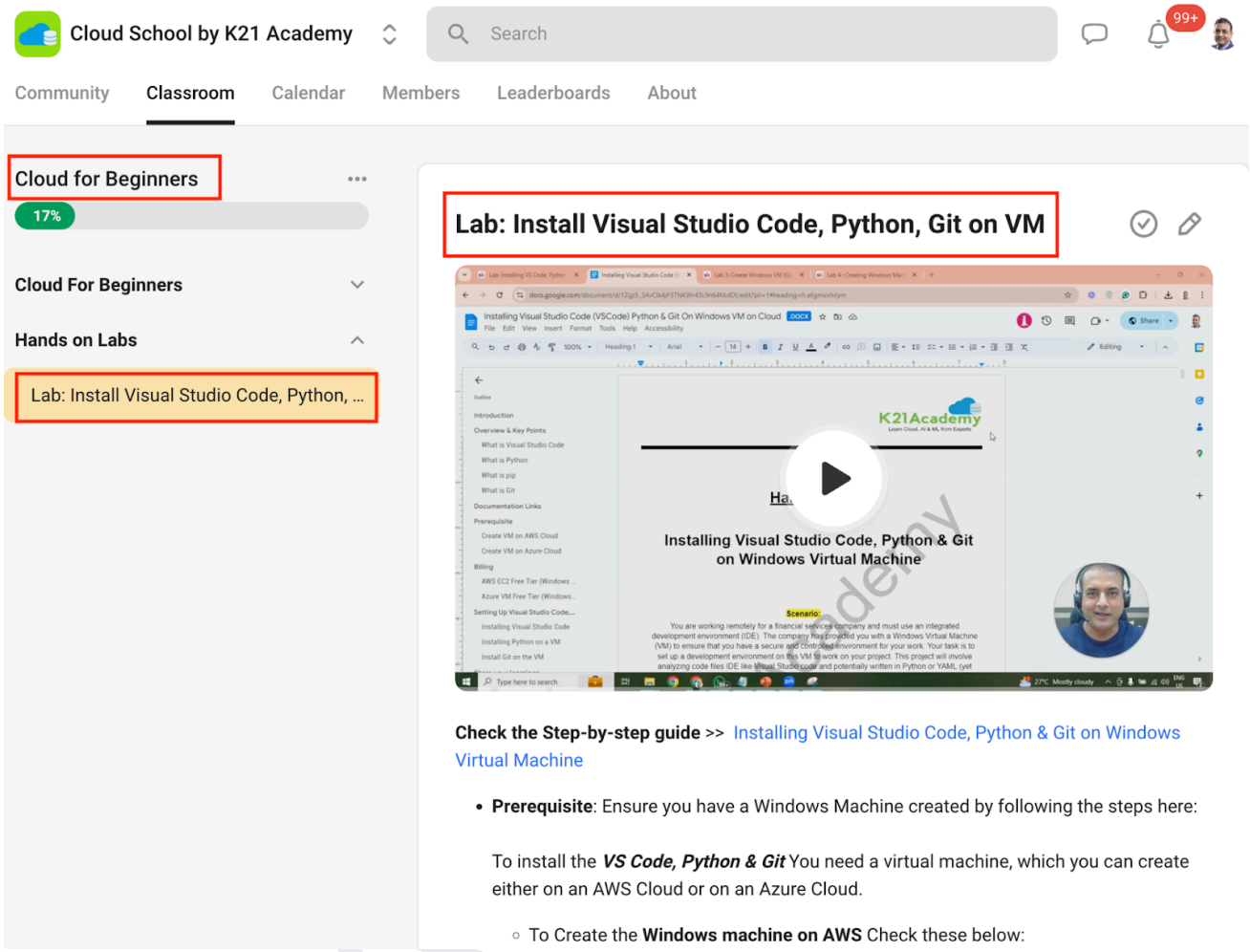
4. PRE-REQUISITE

1. You must have an Azure (Free-Tier or Paid) Account. Follow this Activity Guide inside the Azure for Beginners Course to create a FREE Azure Cloud Account.



2. You must have an Azure VM with Visual Studio Code, Python on it: Check out the Step-by-step Guide Here.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

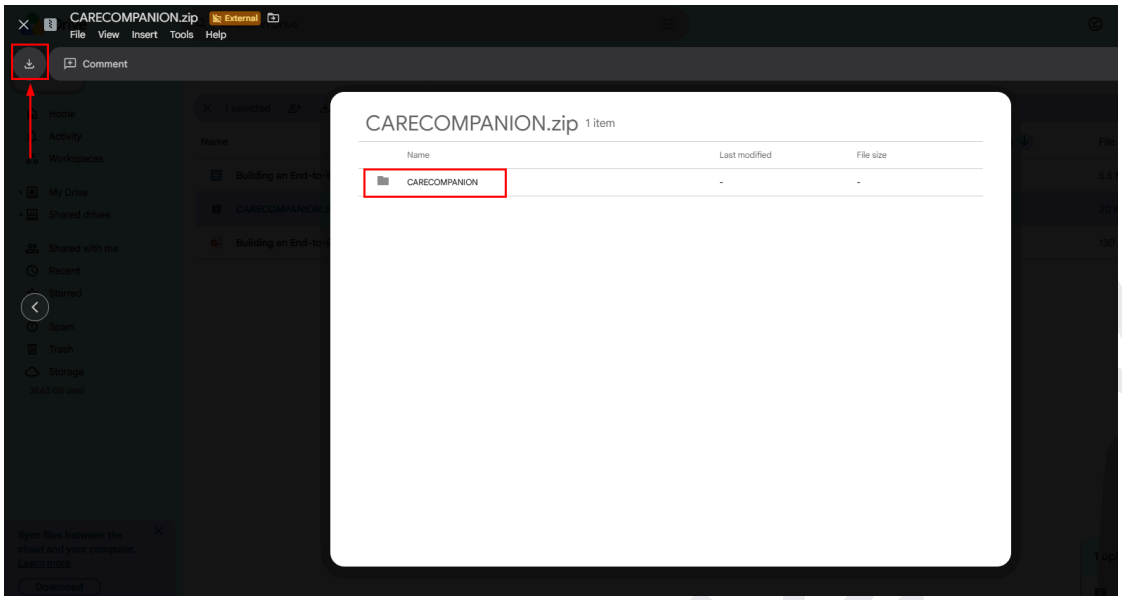


Note: You can also install Visual Studio Code, Python & Git on your Desktop or Laptop (*We only support Visual Studio Code (VSCode) installed on Windows VM on the Cloud, We don't support any issues with VS Code on your Laptop/Desktop*).

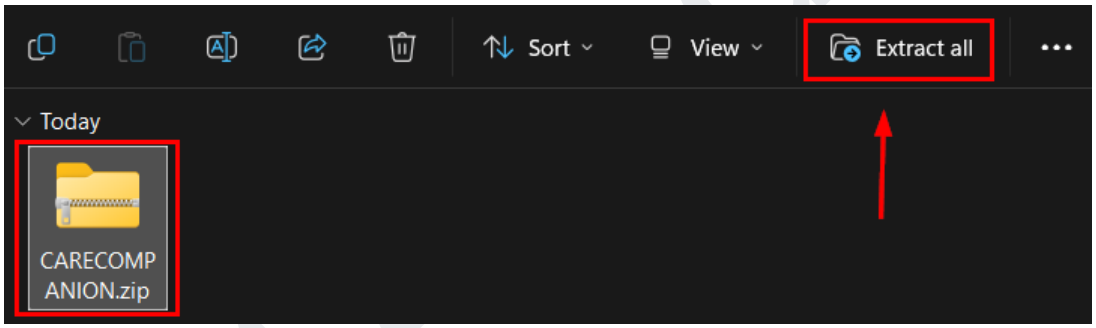
Note: No need to install Git for this lab, just install Python, VS Code and Azure CLI.

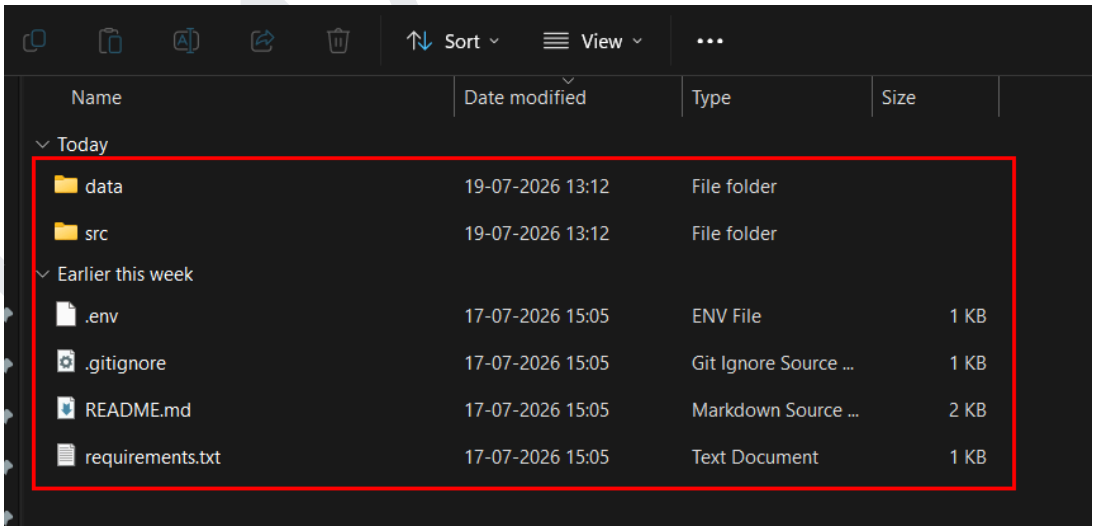
3. Download the CARECOMPANION.zip file and extract the folder inside, we are going to need it in the next steps of the project

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



Extract the files inside the Sample-data.zip folder





Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

5. Cost to Perform the lab

Total Cost: $1–$2

Cost Estimate: Typically $1–$2, depending on model usage, repeated testing, Azure region, and VM runtime.

For the **End-to-End Healthcare AI Assistant with Microsoft Foundry** project, here's the estimated cost for **3–4 hours**:

Microsoft AI Foundry: No direct charge for creating the project or agents. You pay for model tokens and connected tools.

GPT-5-mini and AI Services: Approximately **$0.25–$0.60** for model calls, RAG, multi-agent interactions, embeddings, and discharge-document processing. GPT-5-mini Global pricing is based on input and output token consumption.

RAG Vector Storage: Minimal or no storage cost because File Search includes the first **1 GB of vector storage free**.

Virtual Machine: A Windows 11 Enterprise **Standard_D2s_v3 VM** with 2 vCPUs and 8 GB RAM may cost approximately **$0.33–$0.44 for 3–4 hours**, including a prorated 128-GB Standard SSD. Managed disks are charged hourly while they exist.

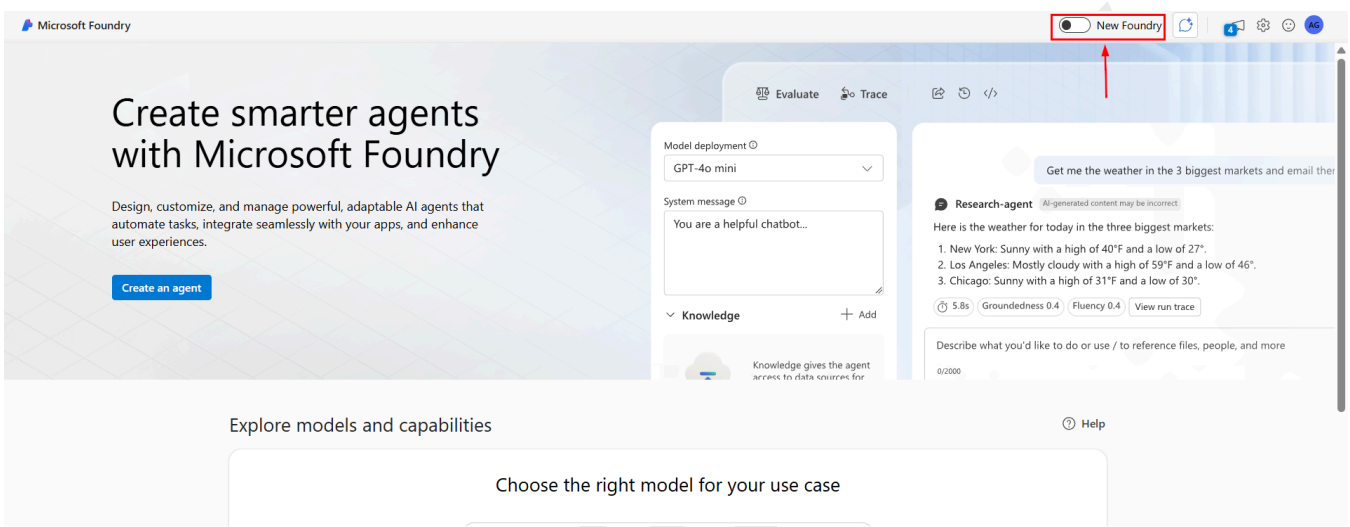
The expected baseline cost is approximately **$0.60–$1.05**, but keeping a **$1–$2 budget** is recommended for additional testing or troubleshooting. This VM estimate assumes an eligible Windows or Microsoft 365 licence for Windows 11 Enterprise.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

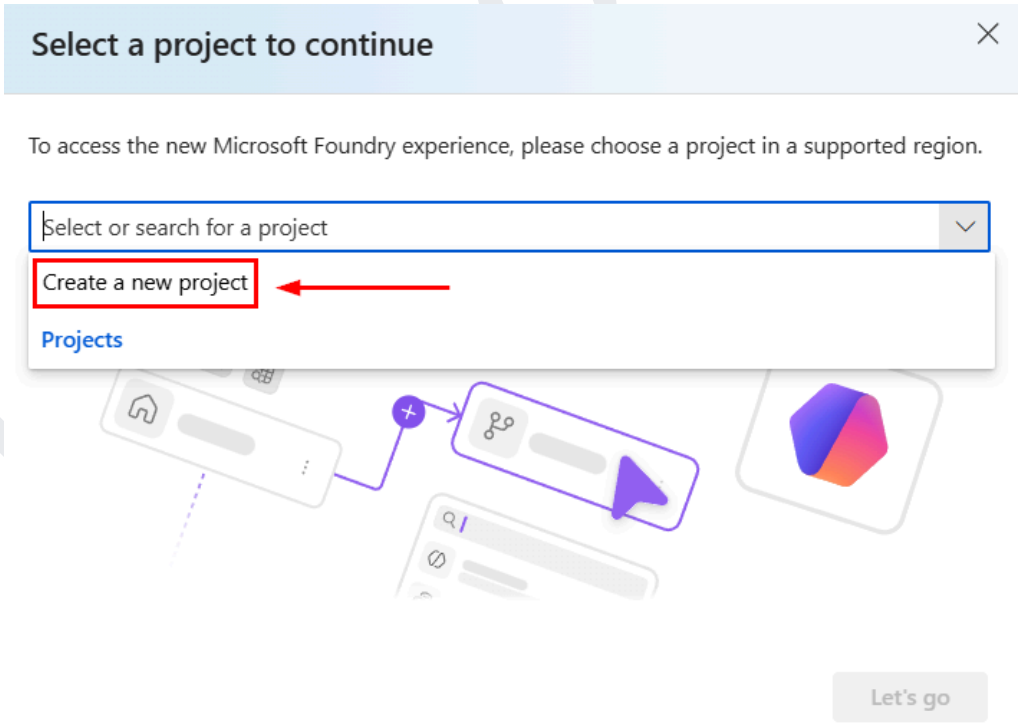
6. Setup Microsoft Foundry and deploy the AI model

In this section, you will create a Microsoft Foundry project, save the required endpoints, and deploy the GPT-5-mini and text-embedding-3-large models for the healthcare AI solution.

1. Open the Microsoft Foundry Portal and make sure to turn on the New Foundry button.

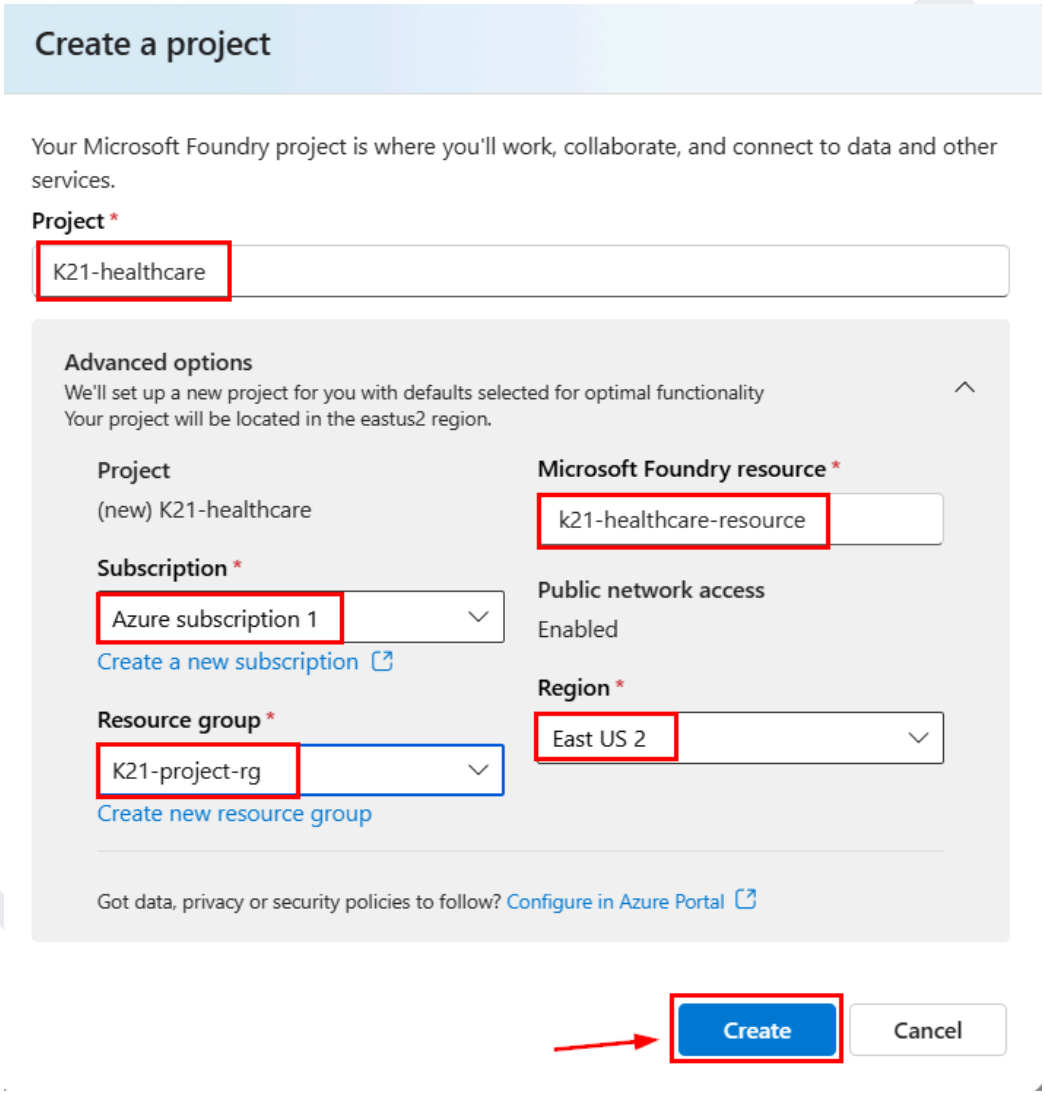


2. Create a new project option.



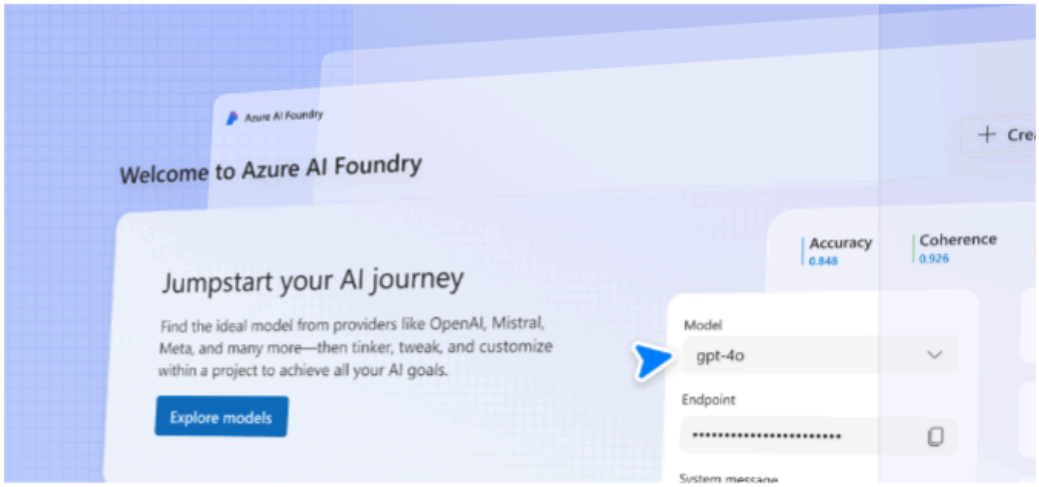
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

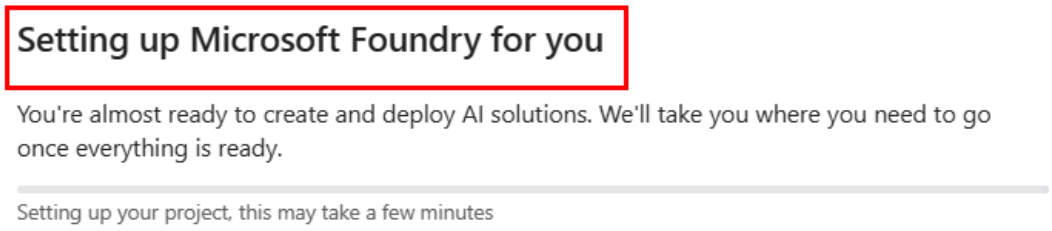
3. Add the following details to create a project:
    a. Project: K21-healthcare
    b. Microsoft Foundry resource: k21-healthcare
    c. Subscription: Choose your Azure subscription
    d. Region: East US 2
    e. Resource group: K21-project-rg (the one wyou have created earlier)



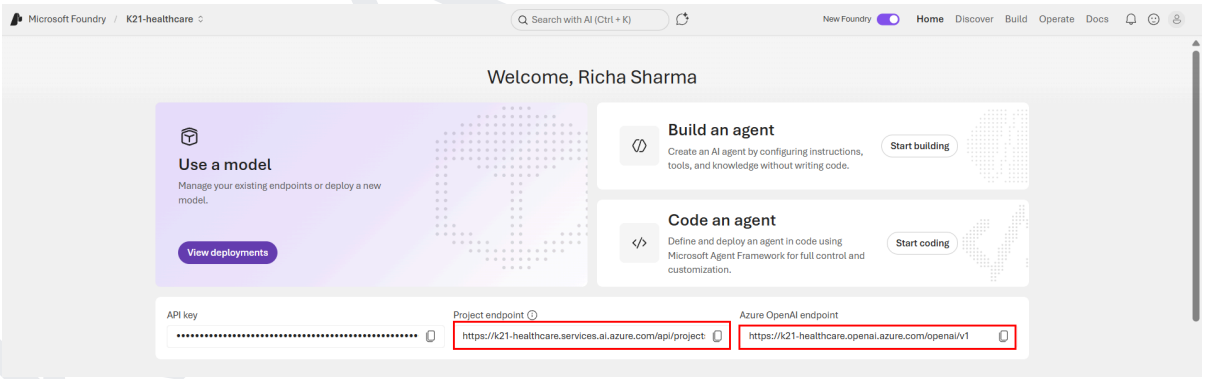
4. Wait for the project to get created.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

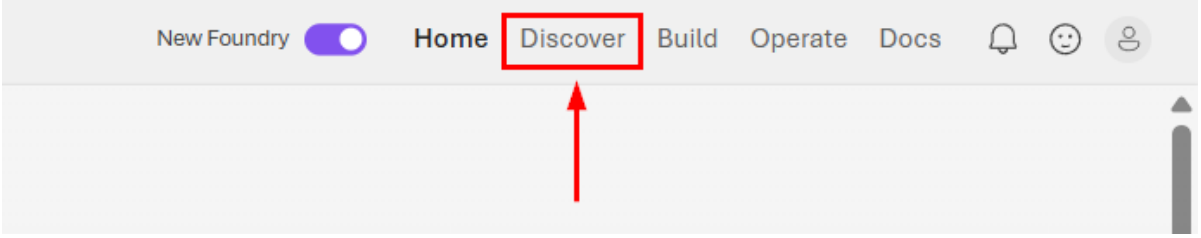




5. After the project has been created, carefully copy the Project endpoint and Azure Open AI endpoint values somewhere safe, we are going to need these values later in the project.

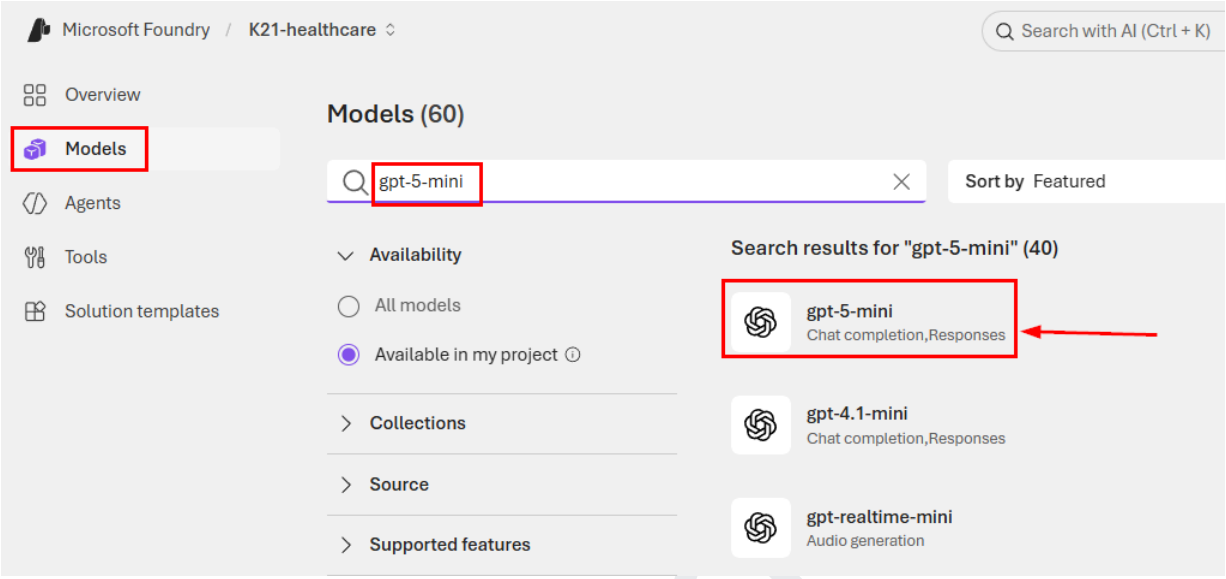


6. Click on Discover on the top right of the project.



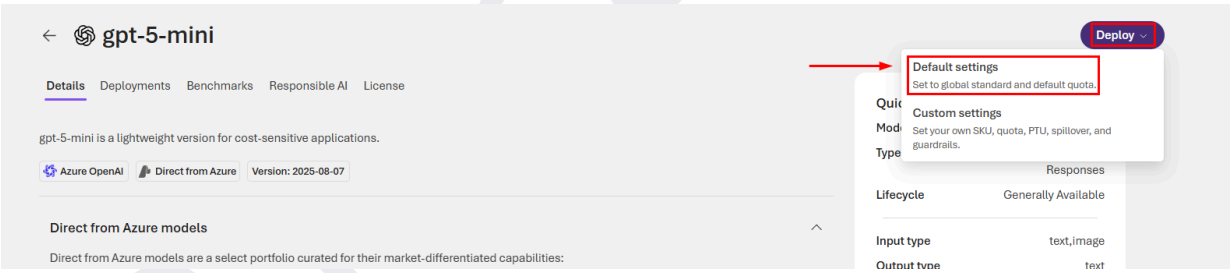
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

7. Choose Models from the left hand menu, and search for gpt-5-mini model.

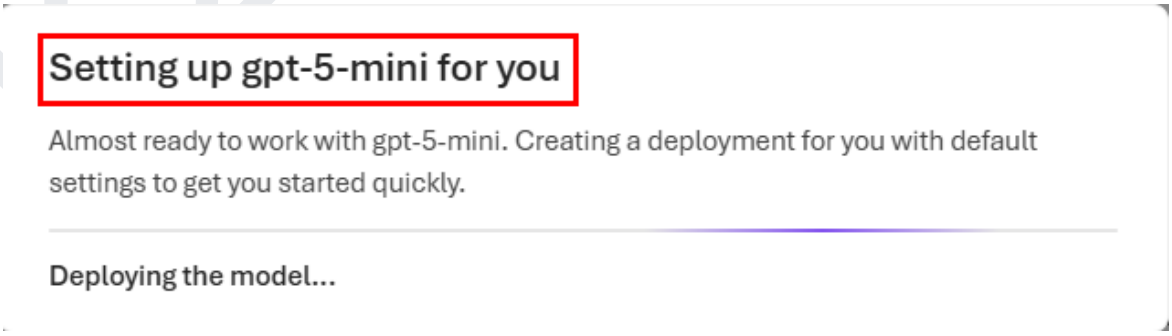


Note: If you don't have the quota for the GPT-5-mini model in a particular region you can choose a different model, any GPT model will work.

8. Click on deploy drop-down and choose the Default settings option.

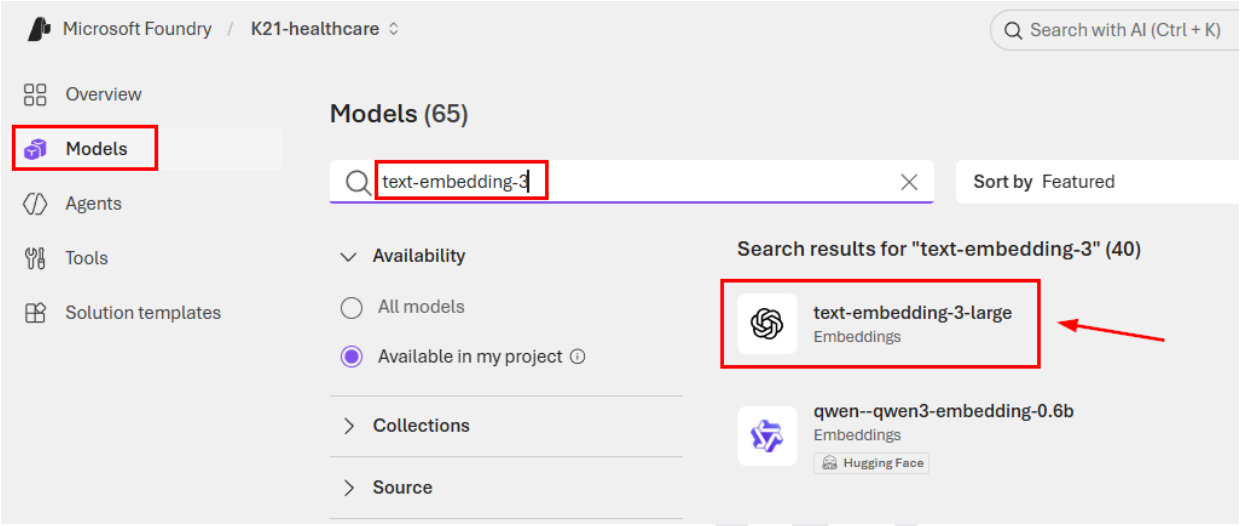


9. Wait for the model to get deployed.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

10. After the model is deployed, it will open in the playground. Now you have to deploy the Text-embedding-3-large model similarly.



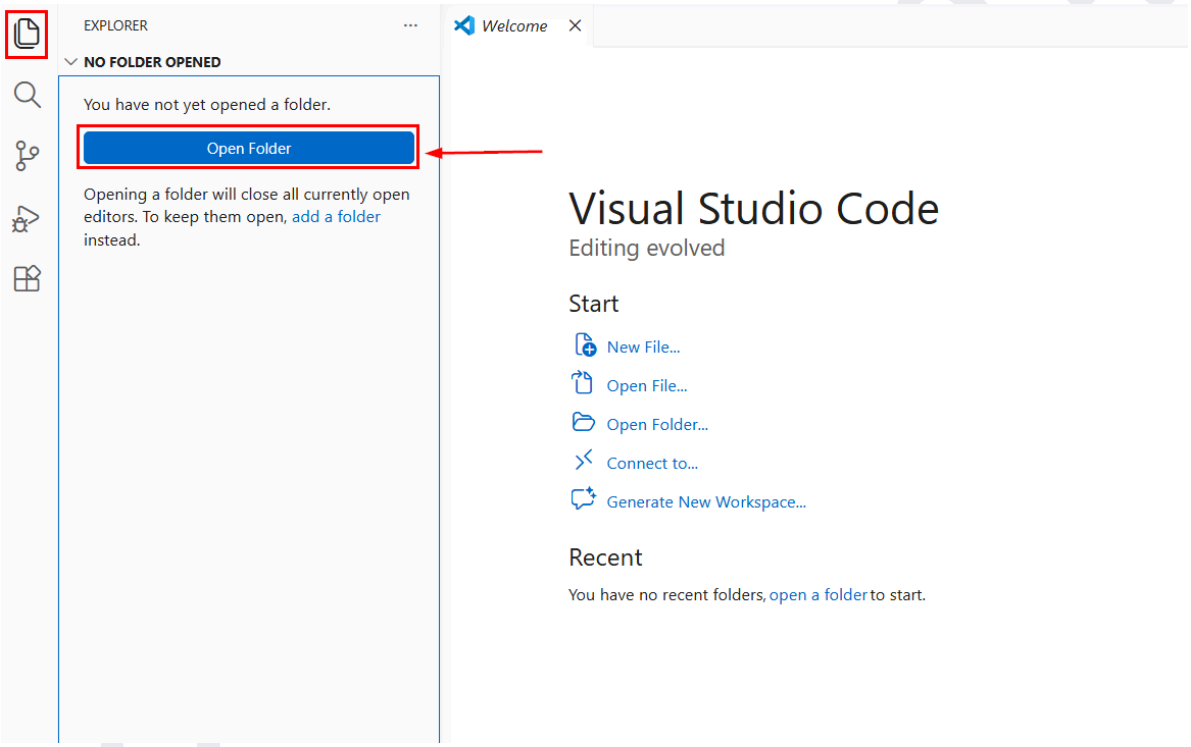
Note: If you don't have the quota for the Text-embedding-3-large model in a particular region you can choose a different model, any Text embedding model will work.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

7. SETUP THE DATA IN VS CODE

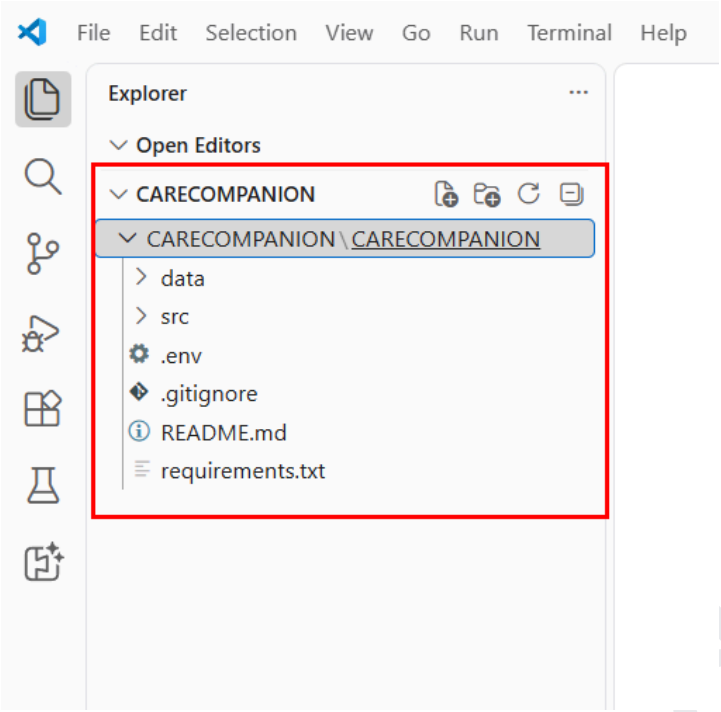
In this section, you will open the project in VS Code, configure the required Foundry endpoints, create the Python virtual environment, install dependencies, and sign in to Azure.

1. Open VS Code and then click on the files icon on the top left on the screen. Then click on Open Folder.

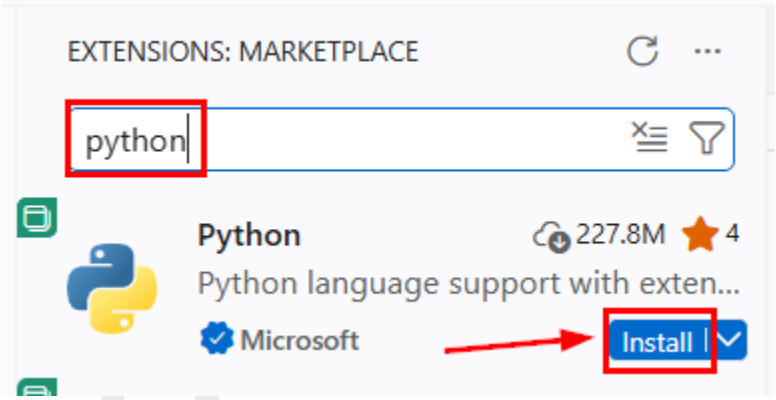


2. Choose the Carecompanion download that you have extracted earlier in the project (in pre-requisite section).

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

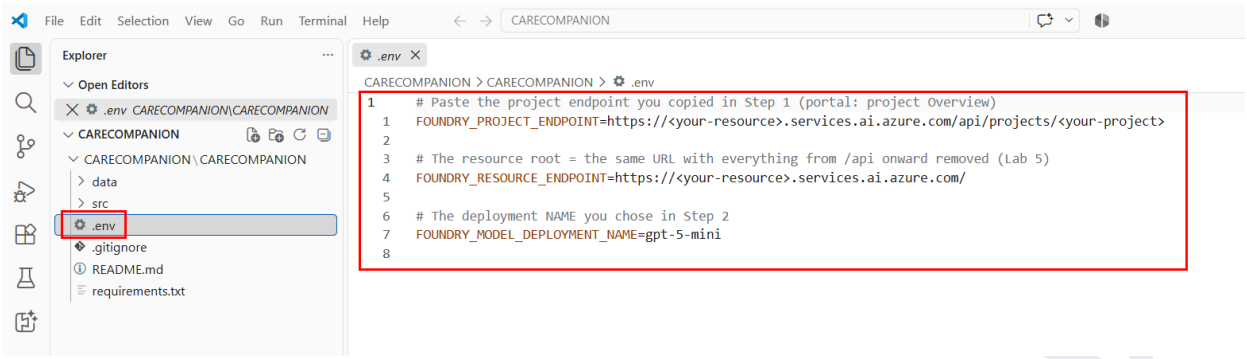


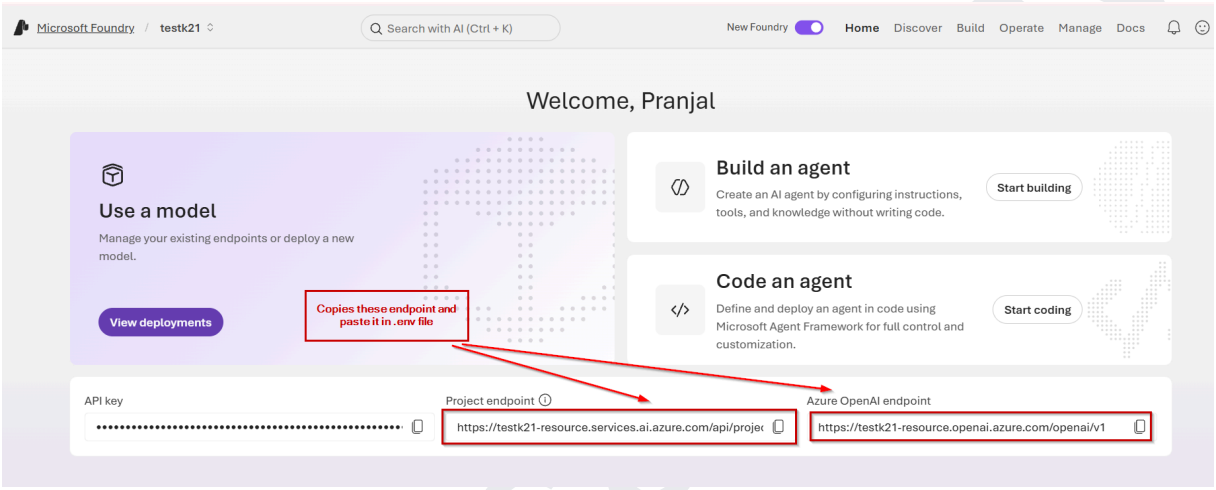
3. Open the extensions option from the left hand side and install the python extension.



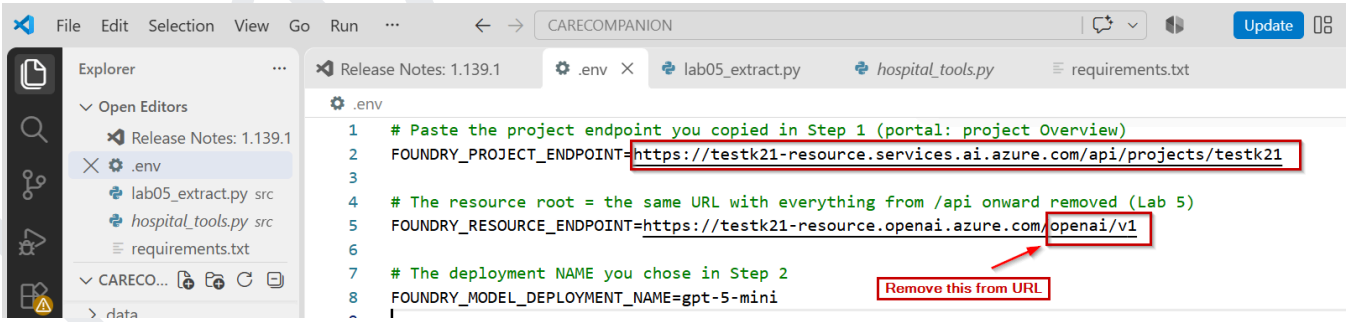
4. Open the .env file and add the Foundry Project endpoint and Foundry Resource endpoint values that we have copied from the Microsoft Foundry portal.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

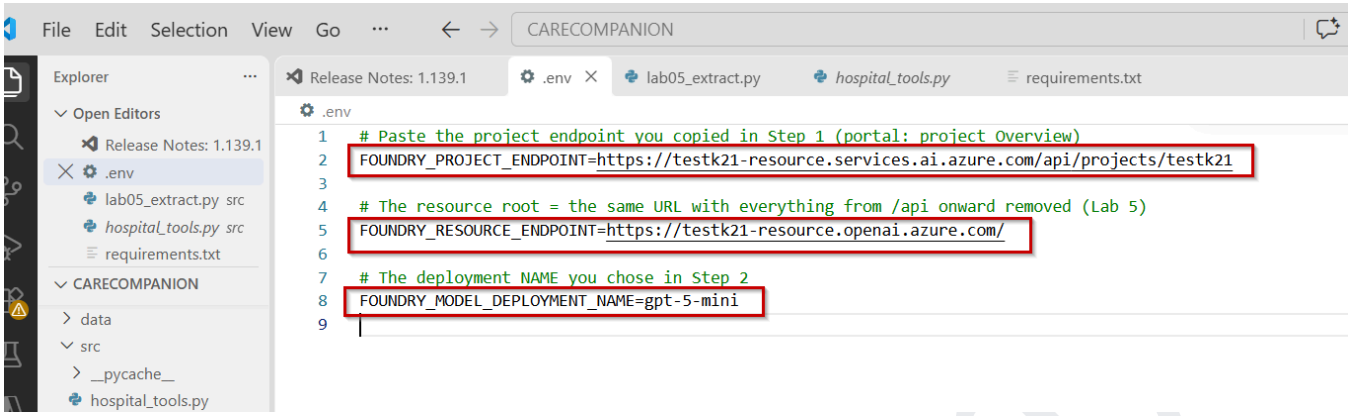




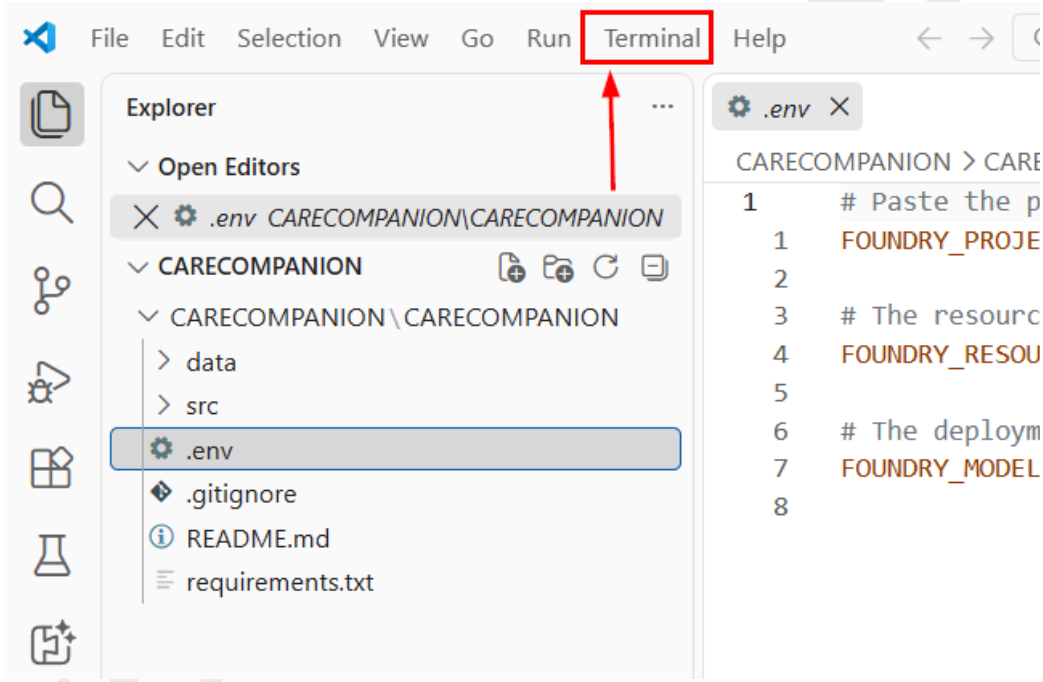
5. Now paste the project endpoint and resource endpoint in the dedicated tags available in the .env file. Make sure to remove the **openai/v1** from the **"Foundry_Resource_Endpoint"** as shown in the screenshot.



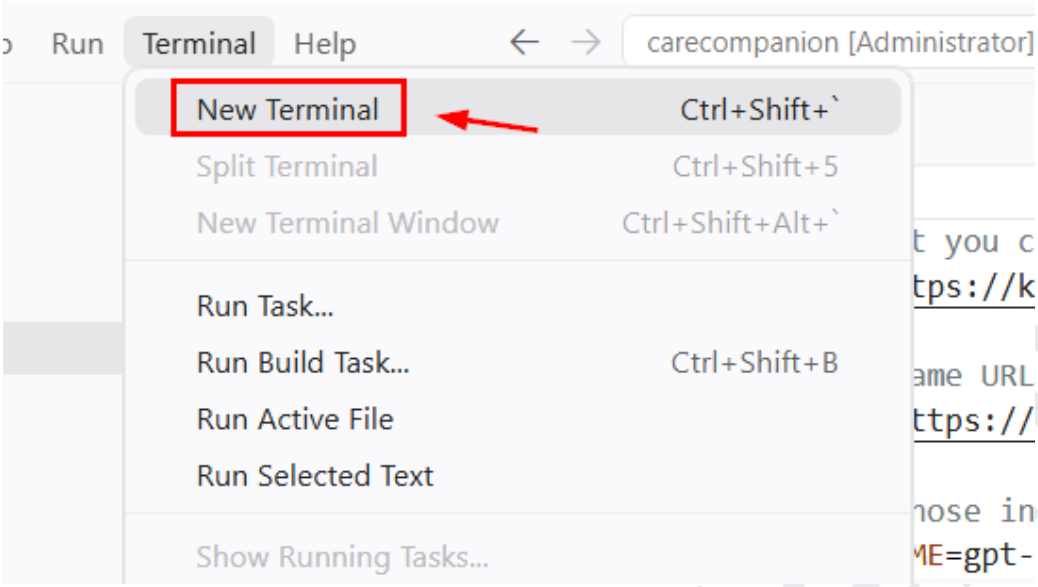
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



6. Open the new terminal by clicking on the terminal button at the top and then choose the new terminal.

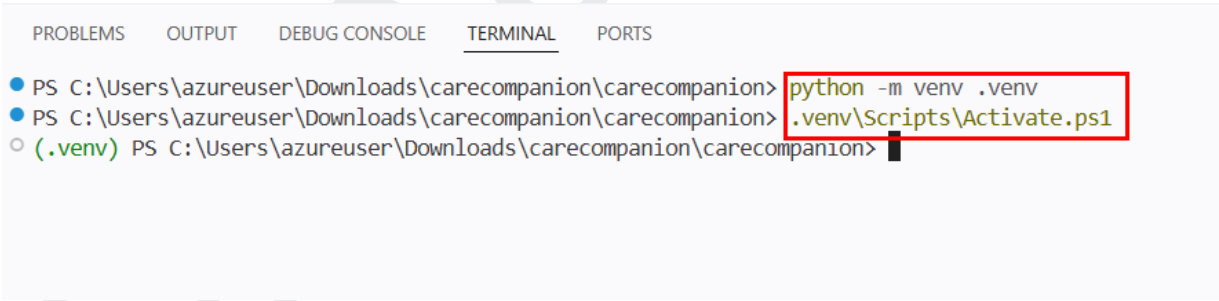


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



7. Run the commands to create a virtual environment

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```



8. Run the command to install the requirements to perform the project.

```
pip install -r requirements.txt
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion> pip install -r requirements.txt
Collecting proto-plus<2.0.0,>=1.24.0 (from google-api-core>=1.26.0->a2a-sdk<2,>=1.0.0->agent-framework-a2a->agent-framework-core[all]==1.11.0->agent-framework>=
1.0.0->-r requirements.txt (line 5))
  Downloading proto_plus-1.28.1-py3-none-any.whl.metadata (2.2 kB)
Collecting google-auth<3.0.0,>=2.14.1 (from google-api-core>=1.26.0->a2a-sdk<2,>=1.0.0->agent-framework-a2a->agent-framework-core[all]==1.11.0->agent-framework>
=1.0.0->-r requirements.txt (line 5))
  Downloading google_auth-2.56.0-py3-none-any.whl.metadata (6.0 kB)
Collecting pyasn1-modules>=0.2.1 (from google-auth<3.0.0,>=2.14.1->google-api-core>=1.26.0->a2a-sdk<2,>=1.0.0->agent-framework-a2a->agent-framework-core[all]==1
.11.0->agent-framework>=1.0.0->-r requirements.txt (line 5))
  Downloading pyasn1_modules-0.4.2-py3-none-any.whl.metadata (3.5 kB)
Collecting pyasn1<0.7.0,>=0.6.1 (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.14.1->google-api-core>=1.26.0->a2a-sdk<2,>=1.0.0->agent-framework-a2a->agent-
framework-core[all]==1.11.0->agent-framework>=1.0.0->-r requirements.txt (line 5))
  Downloading pyasn1-0.6.4-py3-none-any.whl.metadata (8.4 kB)
Collecting ag-ui-protocol<0.2,>=0.1.19 (from agent-framework-ag-ui->agent-framework-core[all]==1.11.0->agent-framework>=1.0.0->-r requirements.txt (line 5))
  Downloading ag_ui_protocol-0.1.19-py3-none-any.whl.metadata (2.9 kB)
Collecting fastapi<0.138.1,>=0.121.0 (from agent-framework-ag-ui->agent-framework-core[all]==1.11.0->agent-framework>=1.0.0->-r requirements.txt (line 5))
  Downloading fastapi-0.138.0-py3-none-any.whl.metadata (27 kB)
Collecting annotated-doc>=0.0.2 (from fastapi<0.138.1,>=0.121.0->agent-framework-ag-ui->agent-framework-core[all]==1.11.0->agent-framework>=1.0.0->-r requiremen
ts.txt (line 5))
```

9. Run the command to login to your Azure account.

```
az login
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No     Subscription name     Subscription ID                       Tenant
-----  --------------------  ------------------------------------  --------------
[1] *  Azure subscription 1  8db44ef1-5785-4968-abb9-05ca0c7dad22  k21academy.com

The default is marked with an *; the default tenant is 'k21academy.com' and subscription is 'Azure subscription 1' (8db44ef1-5785-4968-abb9-05ca0c7dad22).

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: k21academy.com
Subscription: Azure subscription 1 (8db44ef1-5785-4968-abb9-05ca0c7dad22)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.mi
crosoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion>
```

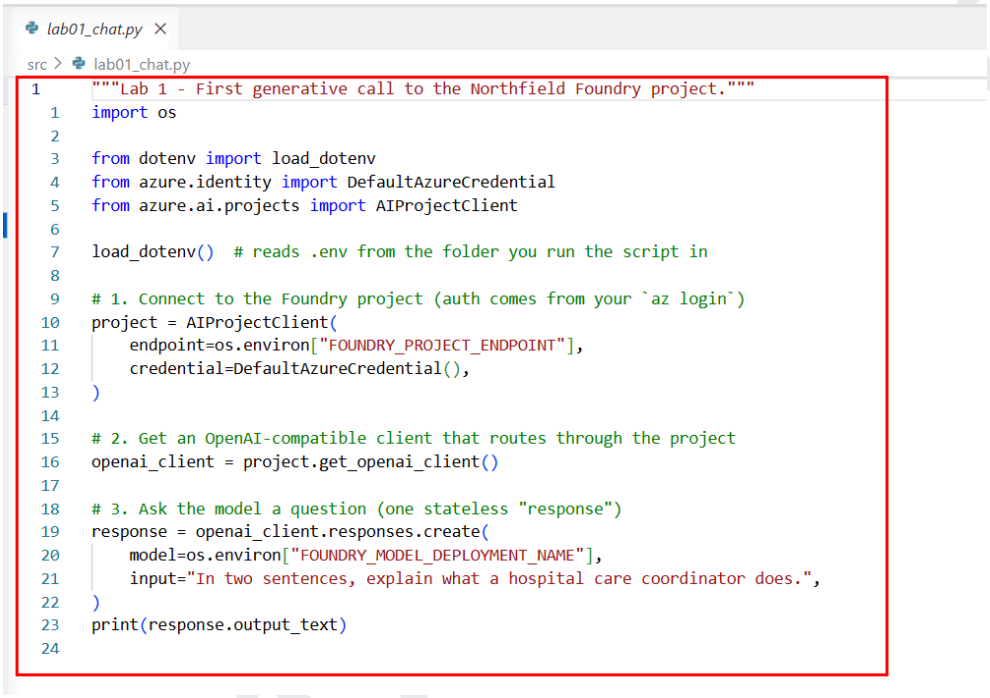
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Ignore

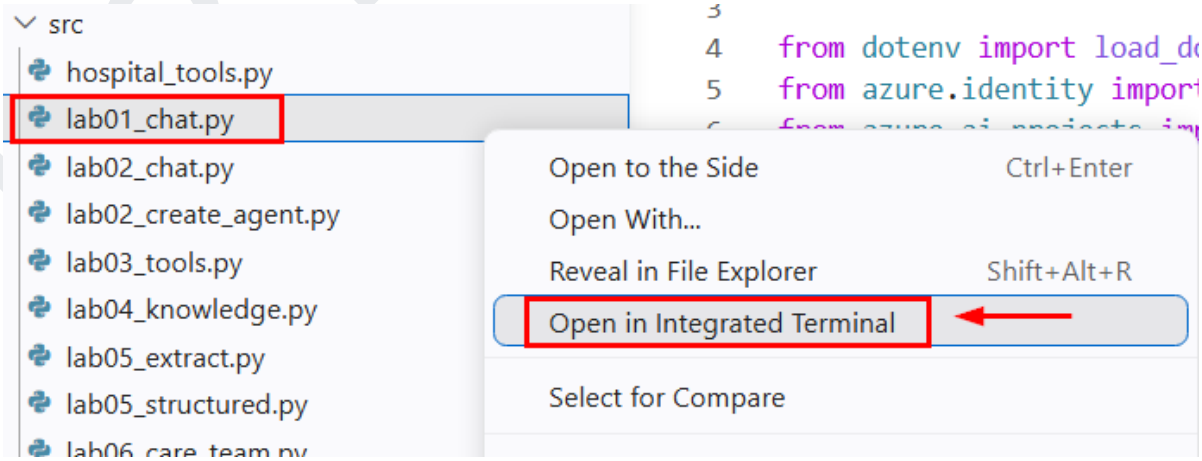
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

8. Build the Healthcare AI Assistant

In this section, you will test the deployed model, create a safety-guided healthcare AI agent, and interact with it through a multi-turn conversation with memory.

1. Open the lab01 file and review the content inside, right click on the lab01_chat.py file and click on open in Integrated terminal.

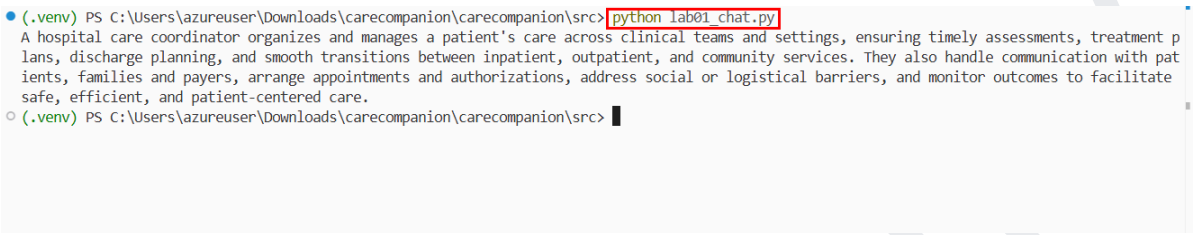




Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

2. Run the file which sends a stateless prompt to the deployed model, and prints the generated response.

```
python lab01_chat.py
```



Confirms that the model connection, authentication, and deployment are working by generating a two-sentence description of a hospital care coordinator.

3. Similarly open the lab02_create_agent.py file and review the content.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

src > lab02_create_agent.py > ...

```
"""Lab 2 - Define (or update) the CareCompanion agent in Foundry."""
import os

from dotenv import load_dotenv
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition

load_dotenv()

INSTRUCTIONS = """You are CareCompanion, the virtual patient-services
assistant for Northfield General Hospital.

Scope: you help with hospital logistics only - visiting hours,
appointments, billing questions, directions, parking, and discharge
paperwork.

Hard rules, in priority order:
1. EMERGENCIES FIRST. If a caller describes anything that sounds like an
   emergency (chest pain, trouble breathing, severe bleeding, stroke
   symptoms), tell them to hang up and call 911 immediately. Do this
   before anything else, every time.
2. NO MEDICAL ADVICE. Never diagnose, interpret symptoms or test
   results, or give medication guidance. If asked, say a clinician must
   answer and offer the nurse advice line: 555-0142.
3. NO GUESSING. If you do not know a hospital-specific fact, say so and
   offer to connect the caller to Patient Services at 555-0100. Never
   invent policies, prices, or schedules.

Style: warm, plain language, short answers. Confirm what the caller
needs before answering if the request is ambiguous.
"""

project = AIProjectClient(
    endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)
```

Connects to Microsoft Foundry and creates a versioned healthcare AI agent using the deployed model and predefined safety instructions.

Note: You can play around with the instructions and make it more fine-tunned according to your own requirements.

4. Run the file lab02_create_agent.py

```
python lab02_create_agent.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab02_create_agent.py
Agent 'carecompanion-agent' is now at version 1
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Confirms that the `carecompanion-agent` was successfully created and saved as version 1.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

5. Now open the lab02_chat.py file and run the file.

```
python lab02_chat.py
```

src > lab02_chat.py > ...

```python
"""Lab 2 - Multi-turn conversation with the CareCompanion agent."""
import os

from dotenv import load_dotenv
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

load_dotenv()

project = AIProjectClient(
    endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)

# The OpenAI client is pre-bound to the agent: every response uses its
# instructions and model automatically.
openai_client = project.get_openai_client(agent_name="carecompanion-agent")

# A conversation = server-side message history for THIS exchange
conversation = openai_client.conversations.create()
print("CareCompanion is on the line. Type 'quit' to hang up.")

while True:
    user_text = input("\nYou: ").strip()
    if user_text.lower() in ("quit", "exit"):
        break
    response = openai_client.responses.create(
        conversation=conversation.id,
        input=user_text,
    )
    print(f"\nCareCompanion: {response.output_text}")
```

```
(.venv) PS E:\k21\CARECOMPANION\CARECOMPANION\CARECOMPANION\src>
(.venv) PS E:\k21\CARECOMPANION\CARECOMPANION\CARECOMPANION\src>    python lab02_chat.py
CareCompanion is on the line. Type 'quit' to hang up.
```

Starts a multi-turn chat with the `carecompanion-agent` and stores the conversation history so the agent can remember earlier messages.

6. Ask some questions to the care companion model.

```
What are the visiting hours?
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab02_chat.py
CareCompanion is on the line. Type 'quit' to hang up.

You: What are the visiting hours?

CareCompanion: Which patient or unit are you asking about (e.g., inpatient room, ICU, nursery, pediatrics)? Visiting hours can vary by unit.

I don't have the hospital's specific visiting-hours policy on hand. I can connect you with Patient Services at 555-0100 who can give the exact hours and any cur
rent exceptions (COVID/other restrictions, special-case rules). If your question is clinical (safe visitors, symptoms, etc.), you can also call the nurse advice
 line at 555-0142.

Which unit should I check for you, or would you like the Patient Services number?

You:
```

My mother is in the hospital and I want to see her tomorrow.

```
You: My mother is in the hospital and I want to see her tomorrow.

CareCompanion: Thanks — I can help with that. When you say "tomorrow" do you mean Sunday, July 18, 2026?

Which unit is your mother on (inpatient floor, ICU, nursery, or pediatrics)? Visiting hours can differ by unit.

I don't have the hospital's unit-specific hours on hand. I can:
- Connect you to Patient Services now at 555-0100 to confirm visiting hours and any visitor restrictions, or
- Give you the number so you can call when convenient.

Would you like me to connect you to Patient Services now? (If your question is clinical — who can visit with symptoms, etc. — you can also call the nurse advice
 line at 555-0142.)

You: quit
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Ln 8, Col 14 Spaces: 4 UTF-8 LF { } Python .venv (3.13.14.final.0)

Confirms that the agent follows its safety instructions, avoids guessing hospital policies, and uses previous conversation context in follow-up responses.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

9. Integrate Hospital Data Using Function Calling

In this section, you will connect the healthcare assistant to patient, appointment, and hospital policy data using function-calling tools and RAG-based file search.

1. Now open the lab03_tools.py file and run it.

src > lab03_tools.py > ...

```python
"""Lab 3 - A tool-using scheduling assistant (Microsoft Agent Framework)."""
import asyncio
import os

from dotenv import load_dotenv
from azure.identity import AzureCliCredential
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient

from hospital_tools import book_appointment, find_patient, get_appointments

load_dotenv()  # the Agent Framework does not load .env on its own


async def main() -> None:
    agent = Agent(
        client=FoundryChatClient(
            project_endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
            model=os.environ["FOUNDRY_MODEL_DEPLOYMENT_NAME"],
            credential=AzureCliCredential(),
        ),
        name="SchedulingAssistant",
        instructions=(
            "You are the scheduling assistant for Northfield General "
            "Hospital. Always use your tools to look up patients and "
            "appointments before answering; never invent records. "
            "Confirm the patient's identity (name -> patient ID) before "
            "booking anything. Never give medical advice."
        ),
        tools=[find_patient, get_appointments, book_appointment],
    )

    session = agent.create_session()  # keeps multi-turn context
    print("Scheduling assistant ready. Type 'quit' to exit.")
    while True:
        user_text = input("\nYou: ").strip()
        if user_text.lower() in ("quit", "exit"):
```

Creates a tool-enabled scheduling agent that uses patient lookup, appointment retrieval, and booking functions while preserving multi-turn context.

```
python lab03_tools.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab03_tools.py
Scheduling assistant ready. Type 'quit' to exit.

You:
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

Does Maria Rivera have any appointments coming up?

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab03_tools.py
Scheduling assistant ready. Type 'quit' to exit.

You: Does Maria Rivera have any appointments coming up?

Assistant: I found Maria Rivera (patient ID: PAT-1001).

She has one upcoming appointment:
- Appointment ID: APT-2002
- Department: Cardiology
- Provider: Dr. A. Chen
- Date & time: 2026-07-29 at 10:00
- Location: Building A, Suite 340
- Status: Confirmed

Would you like me to reschedule, cancel, or book another appointment for Maria?

You:
```

She also needs a lab draw before that visit. Book one for July 24

```
You: She also needs a lab draw before that visit. Book one for July 24

Assistant: Confirmed patient: Maria Rivera (patient ID PAT-1001).

I booked a lab draw for her on 2026-07-24:
- Department: Laboratory
- Date: 2026-07-24
- Confirmation code: NGH-1001-20260724
- Status: Confirmed
- Instructions: Arrive 15 minutes early with photo ID and insurance card.

Would you like me to add a reminder, request a specific time window (morning/afternoon), or do anything else for this appointment?

You: quit
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Confirms that the agent can chain multiple tool calls to identify a patient, check upcoming appointments, and book a new appointment without inventing data.

2. Now open the lab04_knowledge.py file and run it.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

src > lab04_knowledge.py > ...

```
1   """Lab 4 - Ground CareCompanion in the hospital's policy documents."""
1   import os
2   from pathlib import Path
3
4   from dotenv import load_dotenv
5   from azure.identity import DefaultAzureCredential
6   from azure.ai.projects import AIProjectClient
7   from azure.ai.projects.models import FileSearchTool, PromptAgentDefinition
8
9   load_dotenv()
10  POLICY_DIR = Path(__file__).resolve().parent.parent / "data" / "policies"
11
12  INSTRUCTIONS = """You are CareCompanion, the virtual patient-services
13  assistant for Northfield General Hospital.
14
15  Scope: hospital logistics only - visiting hours, appointments, billing,
16  directions, parking, and discharge paperwork.
17
18  Hard rules, in priority order:
19  1. EMERGENCIES FIRST. Anything that sounds like an emergency: tell the
20     caller to hang up and call 911 immediately.
21  2. NO MEDICAL ADVICE. Refer clinical questions to the nurse advice
22     line: 555-0142.
23  3. POLICY ANSWERS COME FROM DOCUMENTS. For any question about hospital
24     policy (hours, billing, discharge), search your files and answer
25     only from what you find, quoting the specific rule. If the documents
26     do not cover it, say so and offer Patient Services at 555-0100.
27
28  Style: warm, plain language, short answers.
29  """
30
31  project = AIProjectClient(
32      endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
33      credential=DefaultAzureCredential(),
34  )
35  openai_client = project.get_openai_client()
36
```

Creates a vector store, indexes the hospital policy documents, and updates the `carecompanion-agent` with file search for grounded RAG responses.

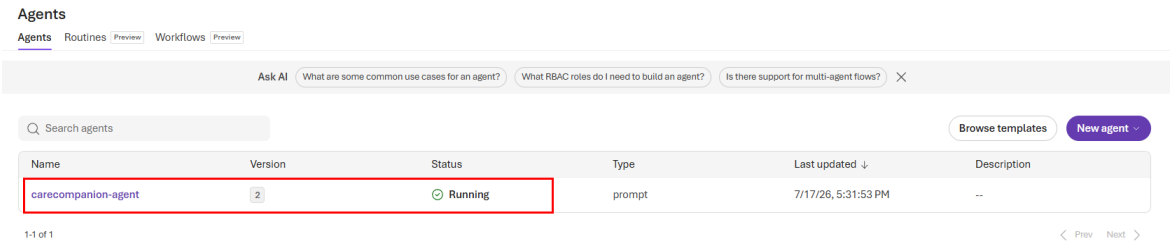
```
python lab04_knowledge.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab04_knowledge.py
Indexed billing_faq.md
Indexed discharge_process.md
Indexed visiting_hours.md
carecompanion-agent is now version 2 (file search attached)

Short answer: no — ICU visiting hours end at 8:00 pm. The ICU visiting policy says visits are 10:00 am to 8:00 pm daily (two visitors at a time; im
mediate family preferred) .

If you think you need an exception or there's an urgent situation, the policy file doesn’t describe after-hours exceptions. Please call Patient Ser
vices at 555-0100 or ask the ICU charge nurse for help.
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission






```
python lab04_knowledge.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab04_knowledge.py
Indexed billing_faq.md
Indexed discharge_process.md
Indexed visiting_hours.md
carecompanion-agent is now version 3 (file search attached)

I couldn't find a hospital policy about validating parking for volunteers in the documents I checked — the Billing FAQ, Visiting Hours, and Discharge Process
 files do not mention volunteer parking validation   .

For a definitive answer, please contact Patient Services at 555-0100.
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Confirms that the policy files were indexed successfully and the agent answered the ICU visiting-hours question using the retrieved hospital policy.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

10. Build the Multi-agent Healthcare Workflow

In this section, you will process discharge documents, extract structured patient information, and build coordinated multi-agent workflows for appointment handling and patient-friendly after-visit summaries.

1. Now open the lab05_extract.py file and run it.

src > lab05_extract.py > ...

```
"""Lab 5 - Read the discharge note with Content Understanding."""
import os
from pathlib import Path

from dotenv import load_dotenv
from azure.identity import DefaultAzureCredential
from azure.ai.contentunderstanding import ContentUnderstandingClient

load_dotenv()
NOTE_PATH = Path(__file__).resolve().parent.parent / "data" / "discharge_note.pdf"

client = ContentUnderstandingClient(
    endpoint=os.environ["FOUNDRY_RESOURCE_ENDPOINT"],  # resource root!
    credential=DefaultAzureCredential(),
)

print(f"Analyzing {NOTE_PATH.name} (this takes ~10-30 seconds)...")
poller = client.begin_analyze_binary(
    analyzer_id="prebuilt-documentSearch",
    binary_input=NOTE_PATH.read_bytes(),
)
result = poller.result()          # SDK polls the long-running job for you

content = result.contents[0]      # one input -> one content item
markdown = content.markdown

out_path = NOTE_PATH.with_suffix(".md")
out_path.write_text(markdown, encoding="utf-8")
print(f"Saved extracted markdown to {out_path}")
print("\n--- first lines ---")
print("\n".join(markdown.splitlines()[:12]))
```

Uses Azure AI Content Understanding to analyze the discharge-note PDF, extract its content as structured Markdown, and save it as a new file.

```
python lab05_extract.py
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab05_extract.py
Analyzing discharge_note.pdf (this takes ~10-30 seconds)...
Saved extracted markdown to C:\Users\azureuser\Downloads\carecompanion\carecompanion\data\discharge_note.md

--- first lines ---
NORTHFIELD GENERAL HOSPITAL
DISCHARGE SUMMARY

Patient: Maria Rivera

MRN: PAT-1001

Date of admission: 2026-07-12

Date of discharge: 2026-07-15

Attending: Dr. A. Chen, Cardiology
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Confirms that the PDF was successfully processed and its headings, patient details, and document structure were preserved in `discharge_note.md`.

2. Now open the lab05_structured.py file and run it.

src > lab05_structured.py > ...

```python
"""Lab 5 - Distill the extracted note into structured JSON."""
import json
import os
from pathlib import Path

from dotenv import load_dotenv
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

load_dotenv()
markdown = (
    Path(__file__).resolve().parent.parent / "data" / "discharge_note.md"
).read_text(encoding="utf-8")

PROMPT = """Extract these fields from the discharge note below and reply
with JSON only (no code fences, no commentary):
- patient_name (string)
- discharge_date (YYYY-MM-DD)
- diagnosis (string)
- medications (array of {name, dose, schedule})
- follow_up ({department, timeframe})
- warning_signs (array of strings)

--- DISCHARGE NOTE ---
"""

project = AIProjectClient(
    endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)
openai_client = project.get_openai_client()
response = openai_client.responses.create(
    model=os.environ["FOUNDRY_MODEL_DEPLOYMENT_NAME"],
    input=PROMPT + markdown,
)

record = json.loads(response.output_text)
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

Reads the extracted Markdown discharge note, sends it to the deployed model, and converts the required patient information into structured JSON.

```
python lab05_structured.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab05_structured.py
      "name": "Clopidogrel",
      "dose": "75 mg",
      "schedule": "once daily (do not miss doses)"
    },
    {
      "name": "Atorvastatin",
      "dose": "40 mg",
      "schedule": "once nightly"
    },
    {
      "name": "Metoprolol succinate",
      "dose": "25 mg",
      "schedule": "once daily"
    }
  ],
  "follow_up": {
    "department": "Cardiology (Dr. Chen)",
    "timeframe": "2 weeks"
  },
  "warning_signs": [
    "Chest pain or pressure",
    "Shortness of breath",
    "Bleeding or swelling at the wrist access site",
    "Fainting"
  ]
}
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Confirms that key details such as the patient name, diagnosis, medications, follow-up plan, and warning signs were successfully extracted.

3. Now open the lab06_care-team.py file and run it.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

src > lab06_care_team.py > ...

```python
"""Lab 6 - A coordinator agent that consults specialist agents."""
import asyncio
import os

from dotenv import load_dotenv
from azure.identity import AzureCliCredential
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient

from hospital_tools import book_appointment, find_patient, get_appointments

load_dotenv()


def make_client() -> FoundryChatClient:
    return FoundryChatClient(
        project_endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
        model=os.environ["FOUNDRY_MODEL_DEPLOYMENT_NAME"],
        credential=AzureCliCredential(),
    )

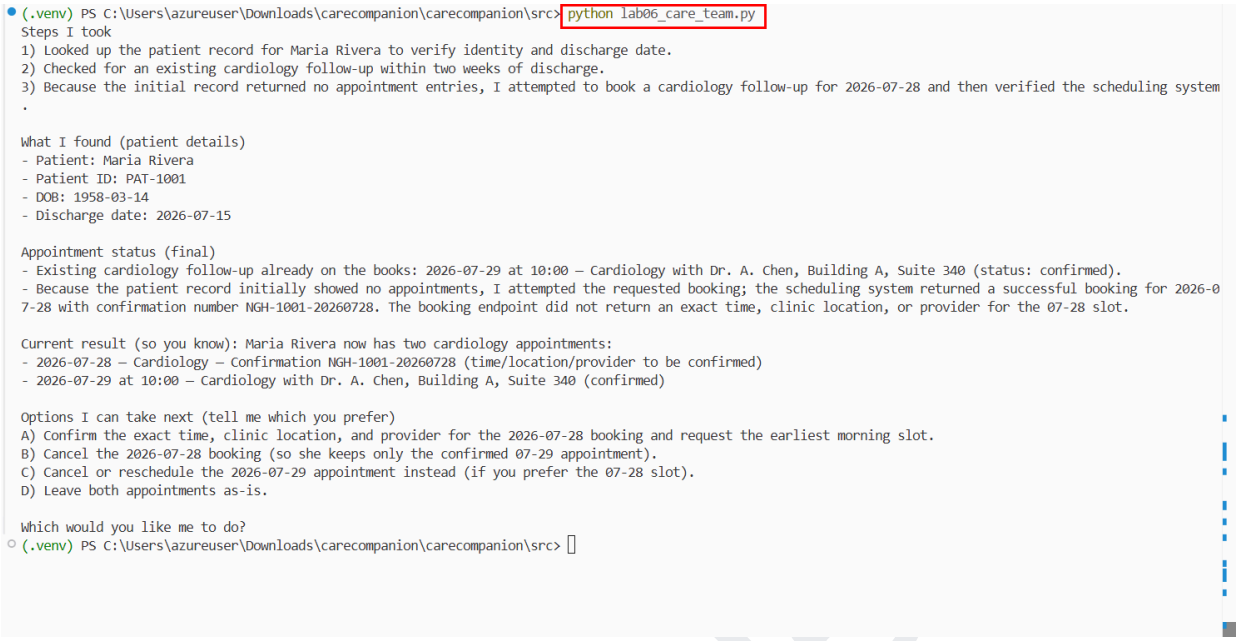

records = Agent(
    client=make_client(),
    name="RecordsSpecialist",
    description="Looks up patient records by name in the registry.",
    instructions=(
        "You look up Northfield General patient records with your tool "
        "and report exactly what you find. Never invent data."
    ),
    tools=[find_patient],
)

scheduler = Agent(
    client=make_client(),
    name="SchedulingSpecialist",
    description="Checks and books appointments for a known patient ID.",
```

Creates a coordinator agent that uses records and scheduling specialist agents as tools to check patient details and manage follow-up appointments.

```
python lab06_care_team.py
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



Confirms that Maria Rivera already has a cardiology follow-up within the required timeframe, so no new appointment is booked.

4. Now open the lab06_pipeline.py file and run it.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

src > lab06_pipeline.py > ...

```python
"""Lab 6 - Sequential workflow: discharge note -> after-visit summary."""
import asyncio
import os
from pathlib import Path

from dotenv import load_dotenv
from azure.identity import AzureCliCredential
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from agent_framework.orchestrations import SequentialBuilder

load_dotenv()
NOTE_MD = Path(__file__).resolve().parent.parent / "data" / "discharge_note.md"


def make_client() -> FoundryChatClient:
    return FoundryChatClient(
        project_endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
        model=os.environ["FOUNDRY_MODEL_DEPLOYMENT_NAME"],
        credential=AzureCliCredential(),
    )


extractor = Agent(
    client=make_client(),
    name="NoteExtractor",
    instructions=(
        "From the discharge note you are given, produce a terse bullet "
        "list of: diagnosis, each medication with dose and schedule, "
        "follow-up appointments, activity limits, and warning signs. "
        "Facts only - no interpretation, no advice."
    ),
)

writer = Agent(
    client=make_client(),
    name="PatientWriter",
```

Builds a sequential two-agent workflow that first extracts key discharge facts and then rewrites them into a simple, patient-friendly after-visit summary.

```
python lab06_pipeline.py
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab06_pipeline.py
Northfield General Hospital — After-Visit Summary

Patient: Maria Rivera
MRN: PAT-1001
Date of admission: 2026-07-12
Date of discharge: 2026-07-15
Attending: Dr. A. Chen, Cardiology

Diagnosis at discharge:
- Coronary artery disease; status post percutaneous coronary intervention (drug-eluting stent, LAD).

Hospital course:
- Ms. Rivera was admitted with exertional chest pain.
- Angiography showed a proximal LAD lesion, treated with a drug-eluting stent via radial access.
- The procedure was uncomplicated and she remained stable through discharge.

Medications at discharge:
- Aspirin 81 mg — once daily
- Clopidogrel 75 mg — once daily (do not miss doses)
- Atorvastatin 40 mg — once nightly
- Metoprolol succinate 25 mg — once daily

Follow-up:
- Cardiology clinic with Dr. Chen in 2 weeks (appointment to be scheduled).
- Primary care within 1 month.
- Cardiac rehabilitation referral placed.

Return to the Emergency Department or call 911 for:
- Chest pain or pressure
- Shortness of breath
- Bleeding or swelling at the wrist access site
- Fainting

Activity:
- No lifting over 10 lb (4.5 kg) for 5 days.
- Walking encouraged daily.

Questions? Call your care team at 555-0100.
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Produces a clear summary while preserving the original medical facts, adding no new advice, and ending with the hospital contact number.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

11. CONFIGURE MCP

In this section, you will connect the AI agent to the Microsoft Learn MCP server, retrieve current documentation through discovered tools, and validate grounded responses from Microsoft Learn.

1. Now open the lab07_mcp.py file and run it.

src > lab07_mcp.py > ...

```python
"""Lab 7 - An agent that uses the Microsoft Learn MCP server."""
import asyncio
import os

from dotenv import load_dotenv
from azure.identity import AzureCliCredential
from agent_framework import Agent, MCPStreamableHTTPTool
from agent_framework.foundry import FoundryChatClient

load_dotenv()


async def main() -> None:
    # The MCP tool manages a live connection, so it is used as an
    # async context manager: connect, discover tools, disconnect.
    async with MCPStreamableHTTPTool(
        name="microsoft-learn",
        url="https://learn.microsoft.com/api/mcp",
    ) as learn_docs:
        agent = Agent(
            client=FoundryChatClient(
                project_endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
                model=os.environ["FOUNDRY_MODEL_DEPLOYMENT_NAME"],
                credential=AzureCliCredential(),
            ),
            name="ITHelpdeskAgent",
            instructions=(
                "You help Northfield General's IT team with Azure and "
                "Microsoft Foundry questions. Always search the "
                "Microsoft Learn documentation with your tools before "
                "answering, and name the page your answer came from."
            ),
            tools=[learn_docs],
        )
        result = await agent.run(
            "What is a Foundry project endpoint, and where do I find "
            "it in the portal?"
```

Connects an AI agent to the Microsoft Learn MCP server so it can discover and use documentation search tools at runtime.

```
python lab07_mcp.py
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab07_mcp.py
Short answer
- A Foundry project endpoint is the single HTTP URL you use to call Microsoft Foundry APIs (SDKs/Responses API/agents) for a specific project. It looks like:
  https://<resource-name>.services.ai.azure.com/api/projects/<project-name>
- You find it in the Foundry portal on the project's welcome or Project details page — the project home shows a "Project endpoint" field with a copy button.

Source
- Quickstart: Set up Microsoft Foundry resources — Microsoft Learn (shows the endpoint format and where the portal displays the Project endpoint): https://learn
.microsoft.com/azure/foundry/tutorials/quickstart-create-foundry-resources#deploy-a-model

If you want, I can give exact step-by-step clicks with screenshots (or show how to extract the base URL or ARM resource ID).
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src>
```

Returns a documentation-grounded explanation of the Foundry project endpoint and identifies where to find it in the portal.

1. Now run the following commands.

```
python hospital_tools.py
```

```
python lab03_tools.py
```

```
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python hospital_tools.py
(.venv) PS C:\Users\azureuser\Downloads\carecompanion\carecompanion\src> python lab03_tools.py
Scheduling assistant ready. Type 'quit' to exit.

You: Book an appointment for Cardiology for K21 Academy

Assistant: I could not find a patient record for "K21 Academy" in the hospital registry.

Please confirm how you'd like to proceed — I can:
- Search again if that name might be spelled differently (give alternate spelling or partial name), or
- Book the Cardiology appointment if you provide the existing patient's full name or patient ID (and I will confirm the matched patient ID first), or
- If this is a new patient, tell me and provide the patient's full name and date of birth so I can create/verify a record (or advise how to create one).

Also tell me the preferred appointment date (YYYY-MM-DD) and any time constraints so I can find an appropriate slot once I have the correct patient.

You: Create an appointment for Maria Rivera on 2026-08-05 for Cardiology

Assistant: Confirmed patient: Maria Rivera (Patient ID: PAT-1001, DOB: 1958-03-14).

I booked the Cardiology appointment for 2026-08-05.

Appointment details
- Patient: Maria Rivera (PAT-1001)
- Department: Cardiology
- Date: 2026-08-05
- Confirmation code: NGH-1001-20260805
- Instructions: Arrive 15 minutes early with photo ID and insurance card.

Existing upcoming appointment on file
- 2026-07-29 at 10:00 with Dr. A. Chen (Cardiology, Building A, Suite 340) — status: confirmed

Do you want me to:
- Cancel or reschedule the 2026-07-29 appointment, or
- Add any notes or an accompanying person to the new 2026-08-05 appointment?

You:
```

```
Create an appointment for Maria Rivera on 2026-08-05 for Cardiology
```

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

```
You: Create an appointment for Maria Rivera on 2026-08-05 for Cardiology

Assistant: Confirmed patient: Maria Rivera (Patient ID: PAT-1001, DOB: 1958-03-14).

I booked the Cardiology appointment for 2026-08-05.

Appointment details
- Patient: Maria Rivera (PAT-1001)
- Department: Cardiology
- Date: 2026-08-05
- Confirmation code: NGH-1001-20260805
- Instructions: Arrive 15 minutes early with photo ID and insurance card.

Existing upcoming appointment on file
- 2026-07-29 at 10:00 with Dr. A. Chen (Cardiology, Building A, Suite 340) — status: confirmed

Do you want me to:
- Cancel or reschedule the 2026-07-29 appointment, or
- Add any notes or an accompanying person to the new 2026-08-05 appointment?

You:
```

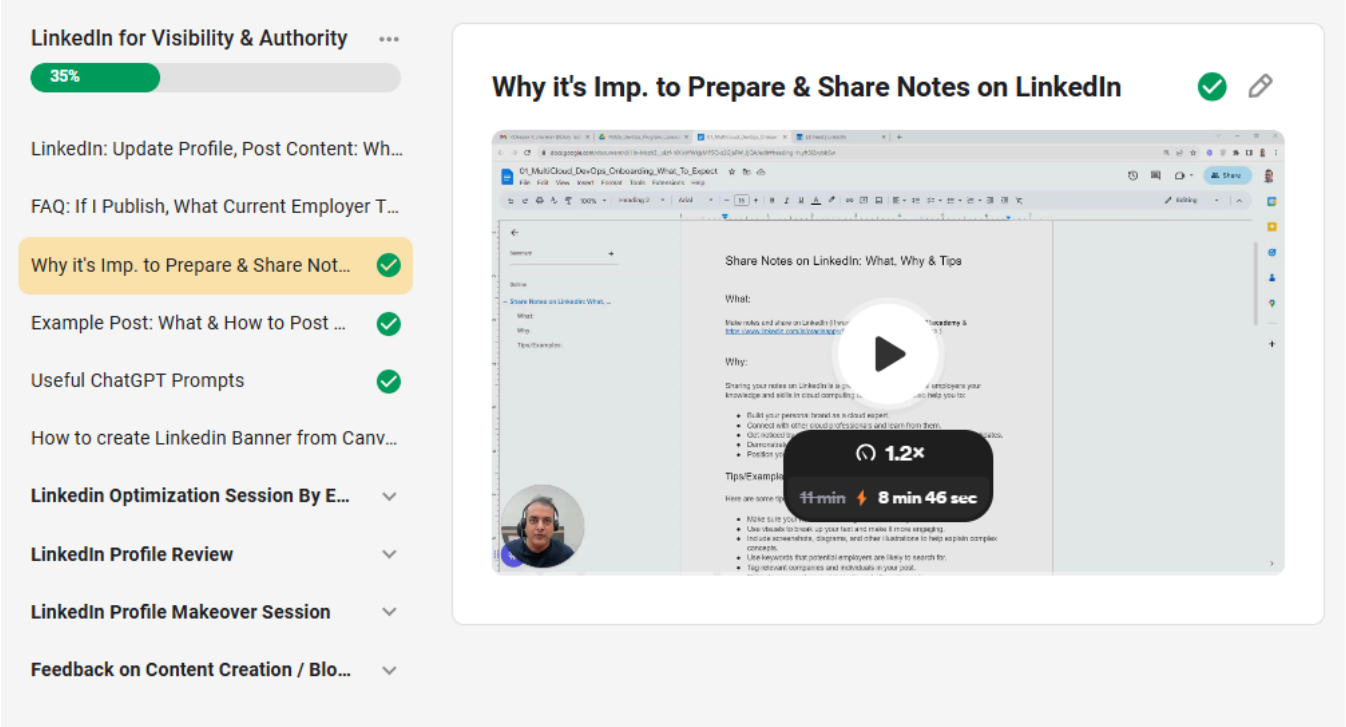
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

12. Share Your Learning on LinkedIn & Community

In this section, you are going to share whatever you learned during this lab on LinkedIn and the Community

Please watch the video below to understand why it's important to post & how to post:

https://www.skool.com/k21academy/classroom/0c24b6af?md=f7975c8e525b4d5d8191e131bdef5740



- As you know, building a strong professional profile is essential in today's job market. One way to showcase your skills and knowledge is by sharing your labs and projects on LinkedIn.

- Your LinkedIn profile is a powerful tool in your job search. Sharing your labs and projects is just one way to demonstrate your expertise and stand out to potential employers. If you don't have LinkedIn, we strongly recommend you create one for yourself.

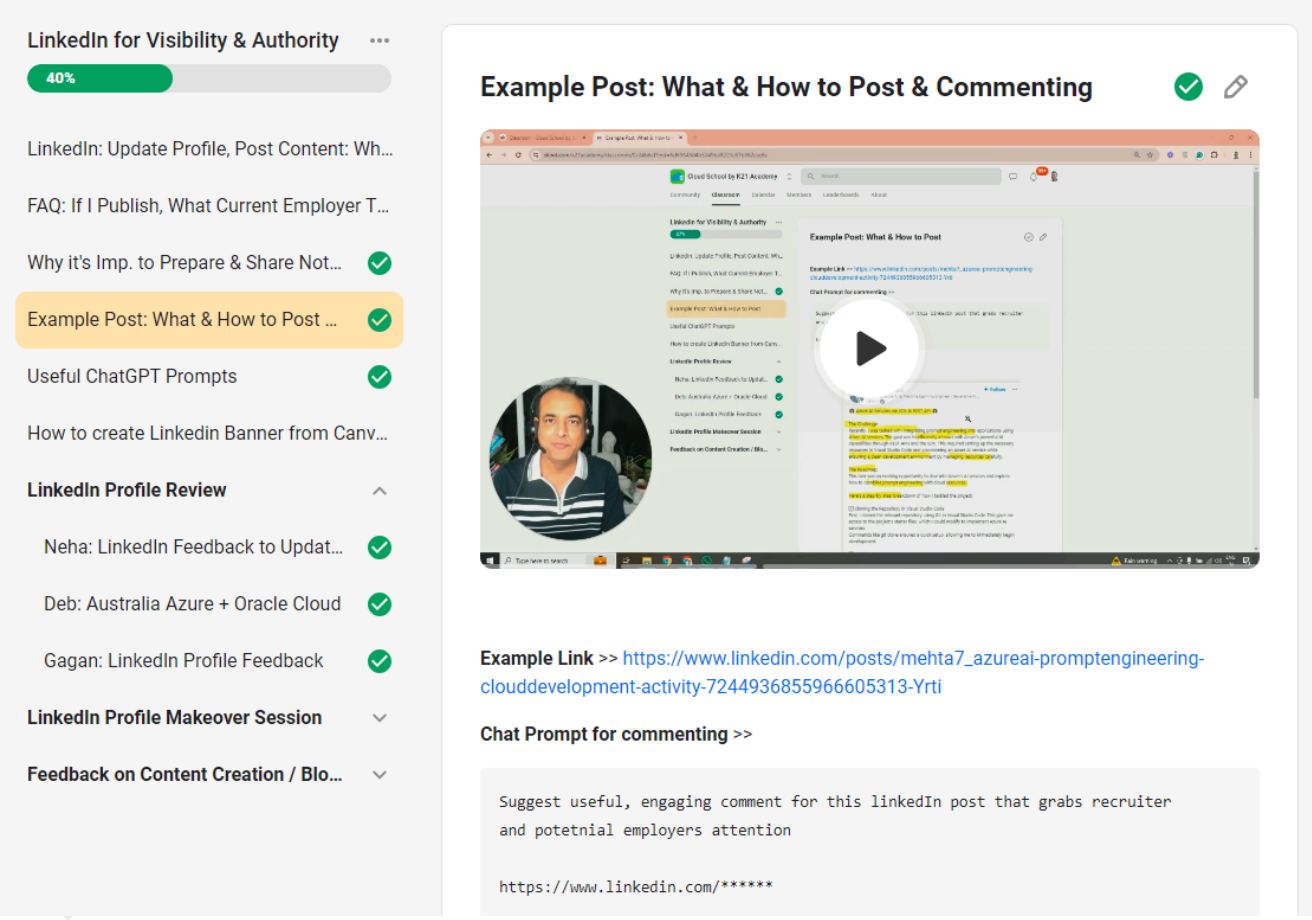
- By doing so, you can demonstrate to potential employers or connections that you have hands-on experience in your field and are actively engaged in learning and growing your skills.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

12.1 Share wins on LinkedIn

Take a screenshot and share it on your LinkedIn. This will attract recruiters and employers to your profile and increase your reach. **Do remember to tag**
K21Academy (https://www.linkedin.com/company/k21academy) &
Atul Kumar (https://www.linkedin.com/in/atulk21academy/), as we will circulate in our network too, to increase your reach.

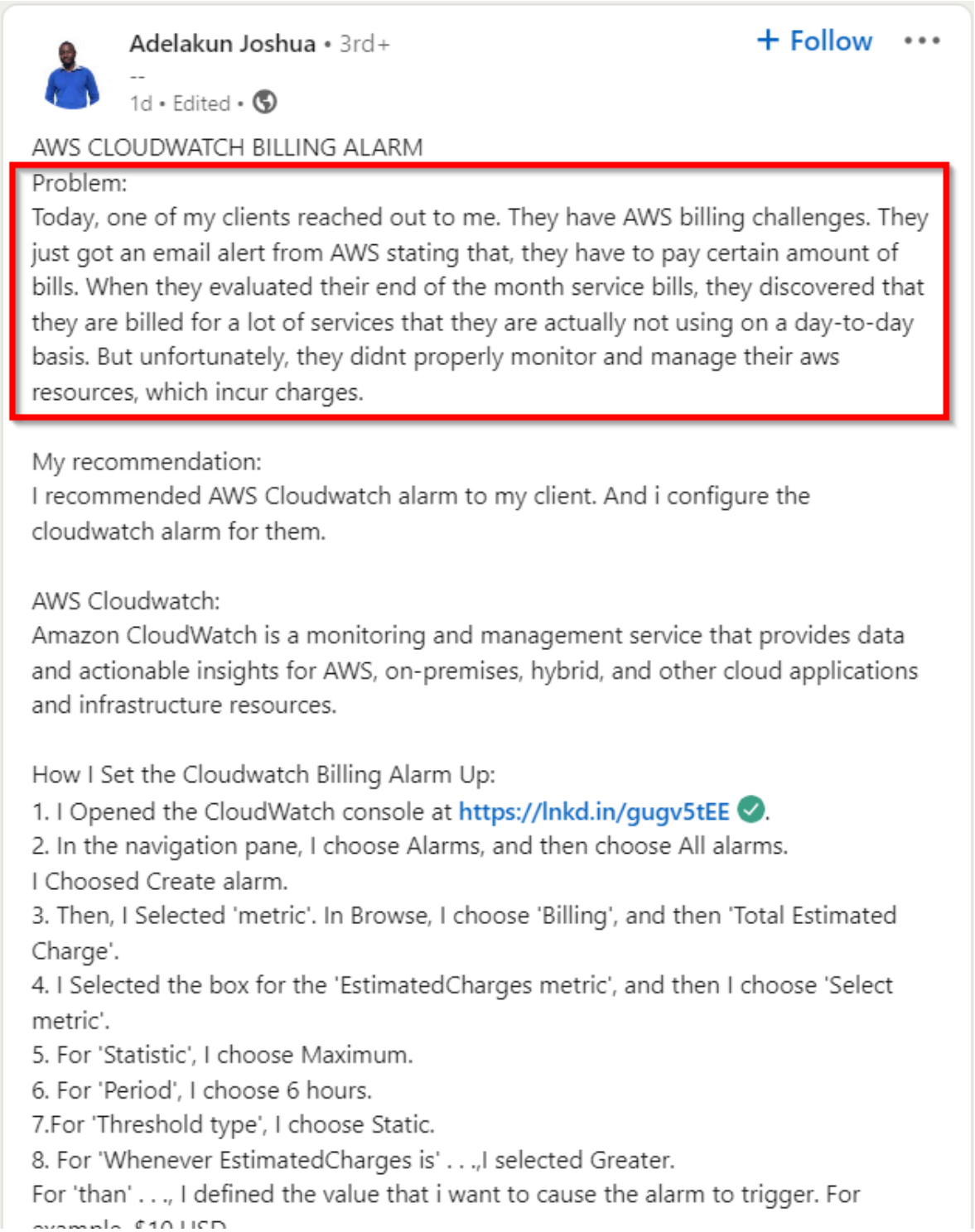
Example Video & Post
https://www.skool.com/k21academy/classroom/0c24b6af



Check this LinkedIn post example
https://www.linkedin.com/posts/izhar-alam-_azure-dp300-database-activity-7061214360462852097-FYQ5/

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

Here is a sample that includes steps inside a post followed by a screenshot:



Adelakun Joshua • 3rd+
--
1d • Edited •

+ Follow •••

AWS CLOUDWATCH BILLING ALARM

Problem:
Today, one of my clients reached out to me. They have AWS billing challenges. They just got an email alert from AWS stating that, they have to pay certain amount of bills. When they evaluated their end of the month service bills, they discovered that they are billed for a lot of services that they are actually not using on a day-to-day basis. But unfortunately, they didnt properly monitor and manage their aws resources, which incur charges.

My recommendation:
I recommended AWS Cloudwatch alarm to my client. And i configure the cloudwatch alarm for them.

AWS Cloudwatch:
Amazon CloudWatch is a monitoring and management service that provides data and actionable insights for AWS, on-premises, hybrid, and other cloud applications and infrastructure resources.

How I Set the Cloudwatch Billing Alarm Up:
1. I Opened the CloudWatch console at https://lnkd.in/gugv5tEE .
2. In the navigation pane, I choose Alarms, and then choose All alarms.
I Choosed Create alarm.
3. Then, I Selected 'metric'. In Browse, I choose 'Billing', and then 'Total Estimated Charge'.
4. I Selected the box for the 'EstimatedCharges metric', and then I choose 'Select metric'.
5. For 'Statistic', I choose Maximum.
6. For 'Period', I choose 6 hours.
7.For 'Threshold type', I choose Static.
8. For 'Whenever EstimatedCharges is' . . .,I selected Greater.
For 'than' . . ., I defined the value that i want to cause the alarm to trigger. For

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

SNS topic included the email address by which my client will recieve email when the billing amount crosses the billing threshold specified.
Note:
You can select an existing Amazon SNS topic, create a 'new Amazon SNS topic', or use a topic ARN to notify other account. If you want your alarm to send multiple notifications for the same alarm state or for different alarm states, choose 'Add notification'.
11. I choosed Next.
12. Under Name and description, I entered a name for the alarm. (Optional) Enter a description of the alarm.
13. Then I Choosed 'Next'.
14. Under 'Preview and create', after confirming that the configuration is correct, and then i choosed 'Create alarm'.

Conclusion: Organization that properly set up a cloudwatch billing alarm, will be able to properly monitor their aws resources usage. The metrics provided by the cloudwatch alarm through the dashboard will also help the organization to effectively control their spending.

#aws
#awscloud
#JTechconsult
#K21Academy: Learn Cloud From Experts
#AtulKumar
#SumtiMehta

Cloudwatch billing alarm architectural diagram:



Other examples:

http://go.k21academy.com/4lSBFcz

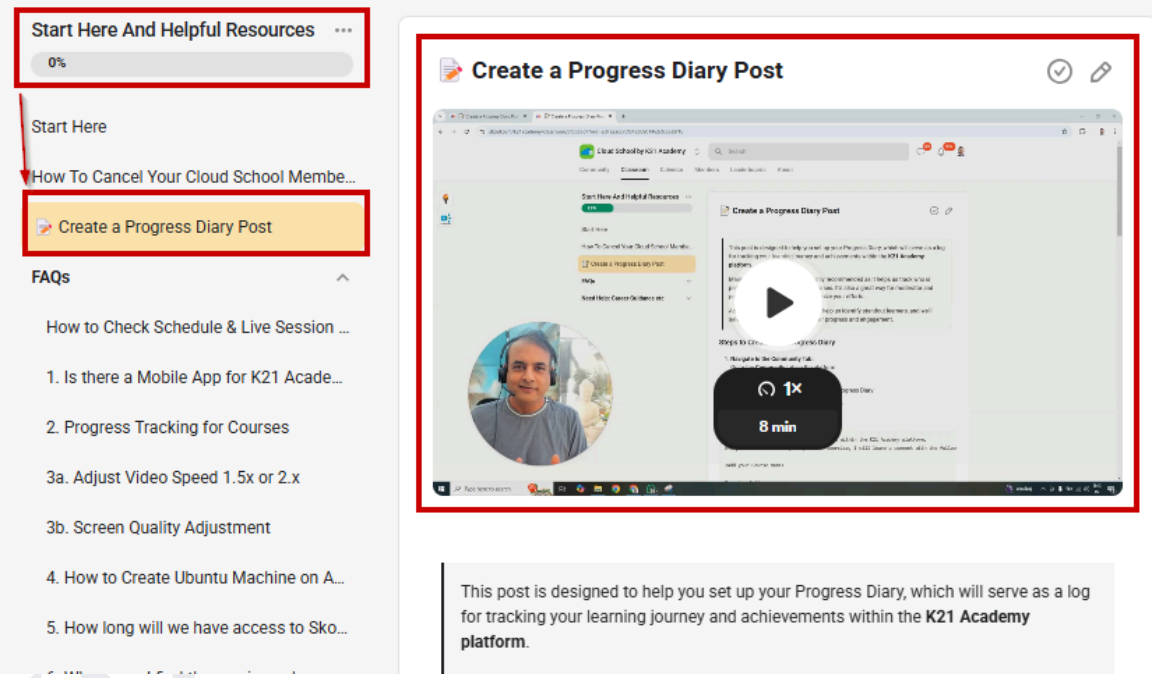
http://go.k21academy.com/4lS1GbD

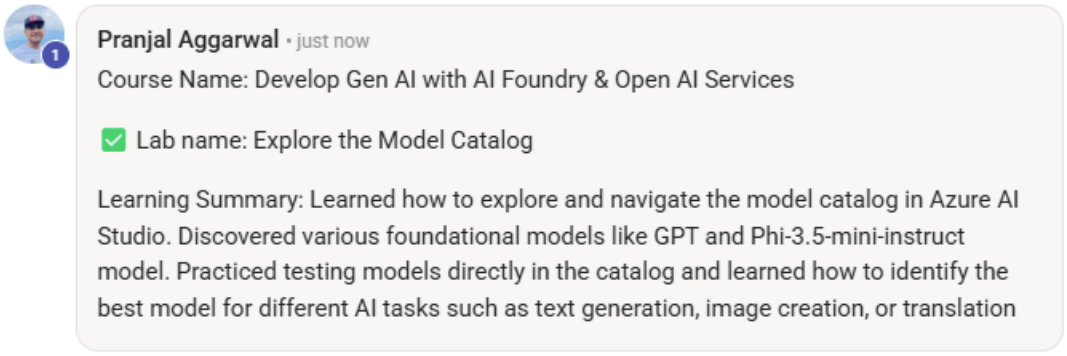
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

12.2 Share the win on the Community via Progress Diary

Share screenshots in Cloud School Community under your **'Progress Diary'** (If you have not created a Progress Diary yet, then check at https://www.skool.com/k21academy/classroom/8f3838d7?md=ad7a2acc7df34280901462cfc65e9f16).

This will boost your confidence in the progressive path you are following, and encourage/inspire others to perform these hands-on labs.





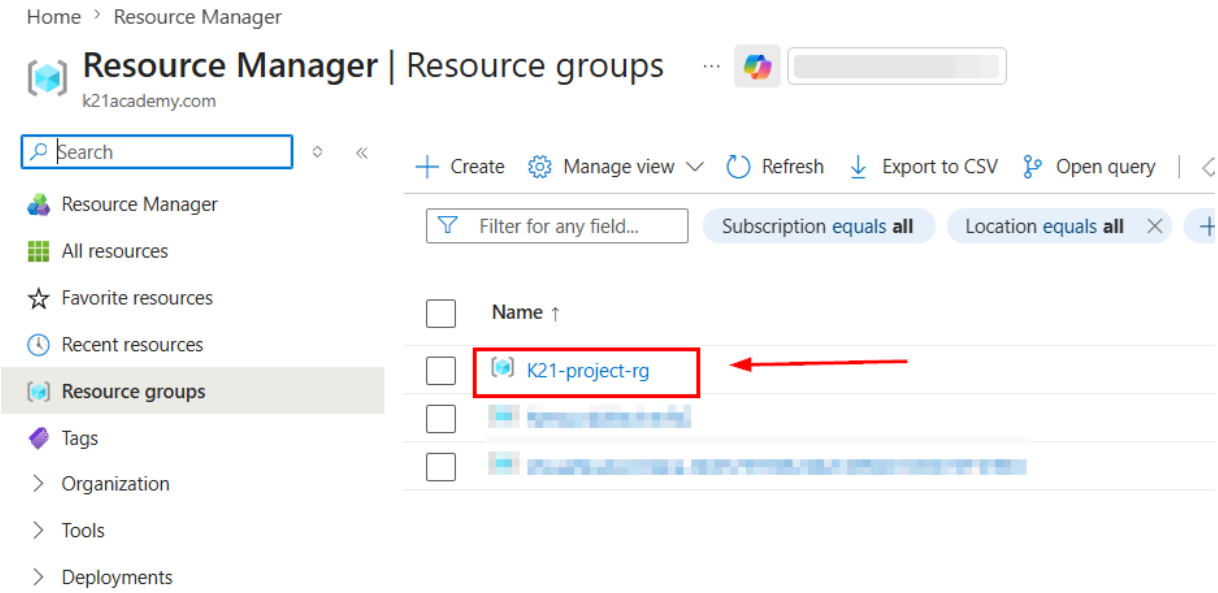
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

13. Delete Azure resources

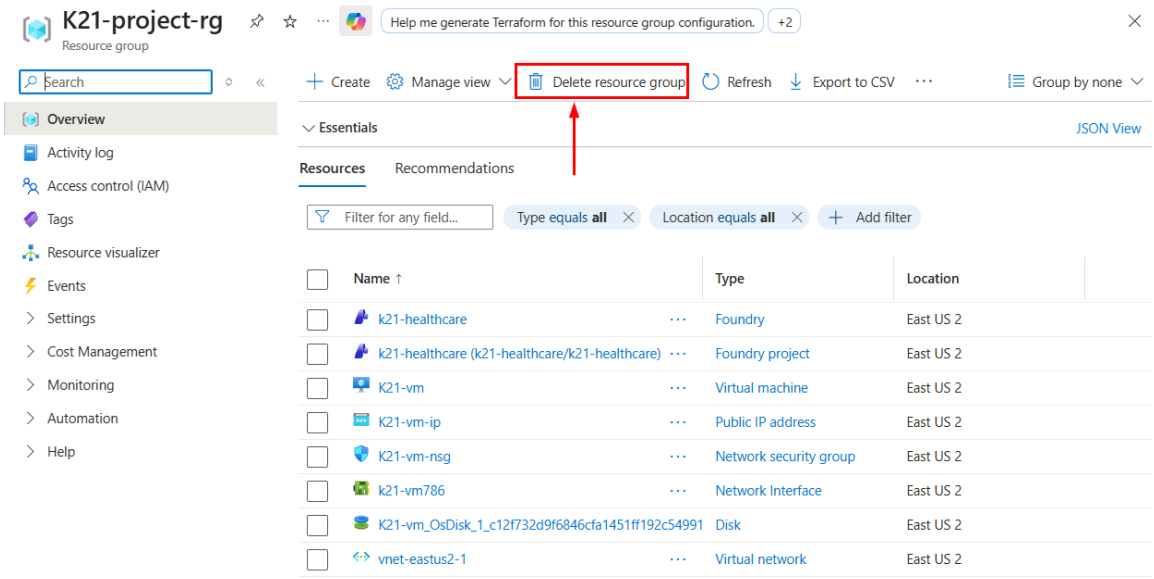
After you complete these steps, you should delete the resources you used to avoid incurring ongoing charges.

By cleaning up these resources, you ensure that you are not charged for their continued use and keep your development environment tidy.

1. When you're done with your project, remember to delete the resources or the entire resource in the Azure portal at https://portal.azure.com.
2. Go to your **resource group** of HousingPricePredictionRG and click on **Delete resource group**

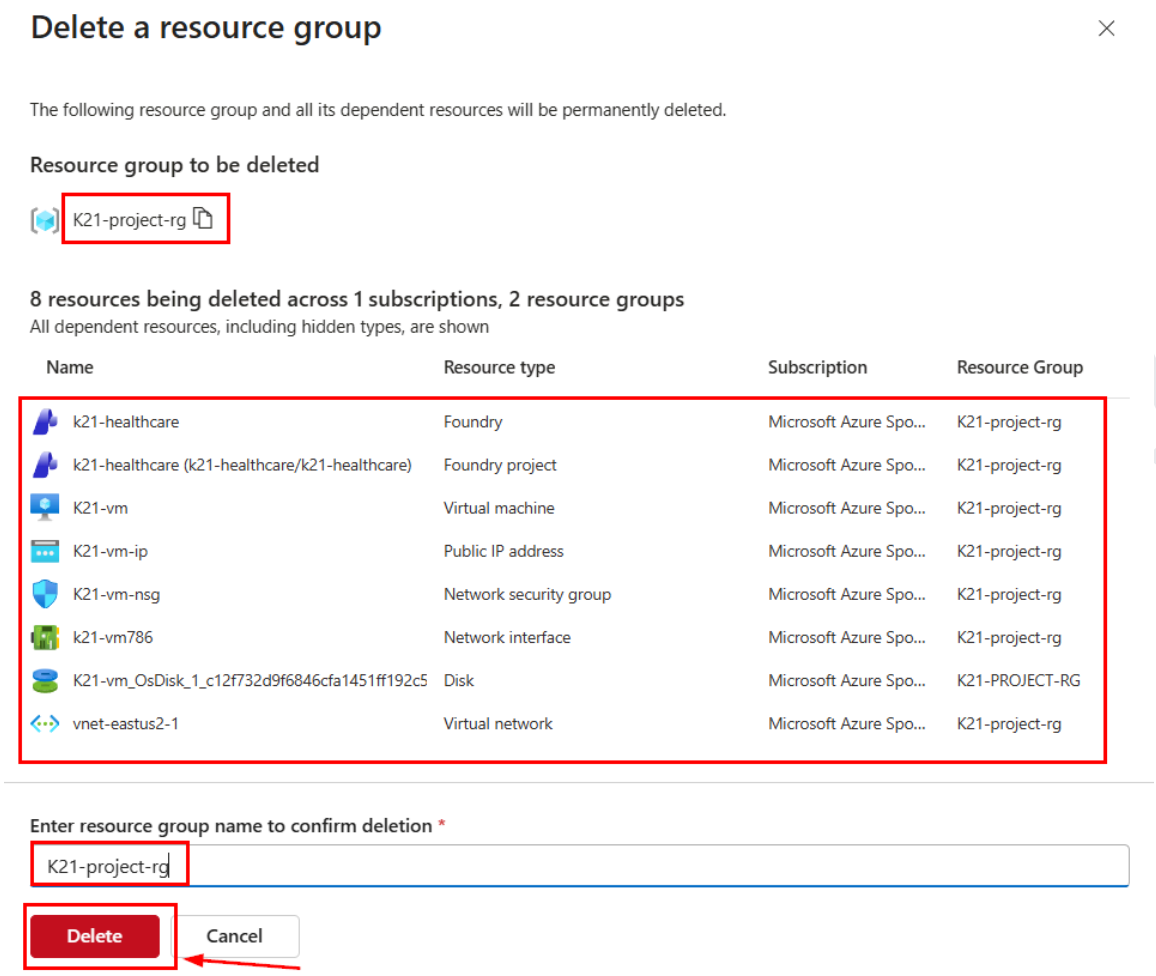


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

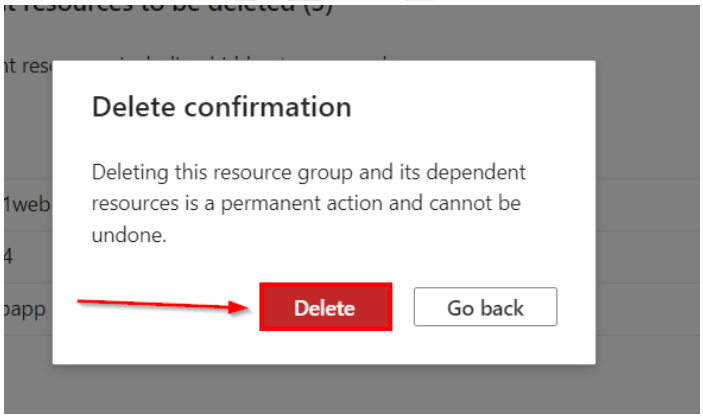


3. On the Delete a resource group pane, enter the resource group name in the area provided and click on **delete**

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission



4. Again click on **Delete.**

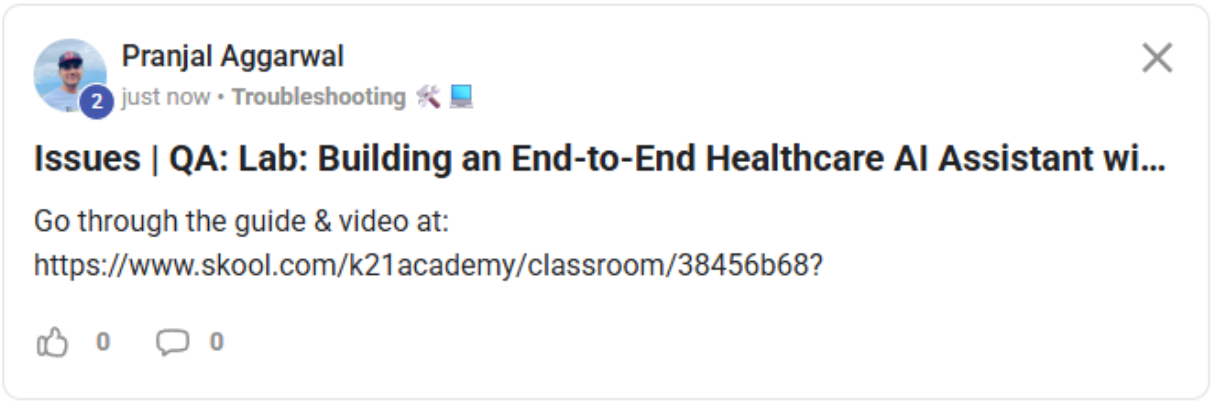


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

14. Troubleshooting Section

Here, we will add some of the common errors encountered while performing the Lab activity.

Note: If you hit any issue in performing this lab, then add your issue in the thread below this lab (linked to this thread in the community): https://www.skool.com/k21academy/issues-qa-lab-building-an-end-to-end-healthcare-ai-assistant-with-microsoft-foundry-post-here



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

15. SUMMARY

In this Project, we completed the following main steps to build an end-to-end healthcare AI assistant:

- **Set Up Microsoft Foundry:** We created a Microsoft Foundry project, copied the required project and resource endpoints, and deployed the GPT-5-mini and text-embedding-3-large models.
- **Configure the Development Environment:** We opened the project in Visual Studio Code, added the Foundry endpoints to the .env file, created a Python virtual environment, installed the required packages, and authenticated with Azure.
- **Build the Healthcare AI Assistant:** We tested the deployed model, created a versioned healthcare agent with emergency and medical-safety instructions, and enabled multi-turn conversations with memory.
- **Integrate Hospital Data Using Function Calling:** We connected the assistant to synthetic patient and appointment data through tools for patient lookup, appointment retrieval, and follow-up booking.
- **Implement RAG with Hospital Policies:** We created a vector store, indexed hospital policy documents, and added file search so the assistant could provide grounded answers based on trusted information.
- **Process Discharge Documents:** We used Azure AI Content Understanding to extract a discharge note into Markdown and converted important patient details into structured JSON.
- **Build Multi-Agent Workflows:** We created specialist agents for patient records and scheduling, coordinated them through a central agent, and built a sequential workflow to generate patient-friendly after-visit summaries.
- **Configure MCP Integration:** We connected an agent to the Microsoft Learn MCP server so it could discover documentation tools and provide answers grounded in current Microsoft documentation.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission

- **Test the Complete Solution:** We validated model connectivity, conversation memory, safety instructions, function calling, policy retrieval, document extraction, and multi-agent coordination.
- **Delete Resources:** After completing the project, we removed vector stores, agents, and the Azure resource group to avoid unnecessary charges.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission